



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**

Membre de 
HONORIS UNITED UNIVERSITIES

PROJET DE FIN DE MODULE

Module : Deep Learning

Conception, Implémentation et Analyse Critique de Modèles de Deep Learning

Réalisé par : Sabri Hiba

Encadrante : Hidila Zineb

Date de remise : 26/06/2026

Remerciements

La réalisation de ce projet constitue une étape importante dans mon parcours académique et n'aurait pas été possible sans le soutien, l'accompagnement et les conseils de plusieurs personnes auxquelles je souhaite exprimer ma profonde gratitude. Je tiens tout particulièrement à adresser mes plus sincères remerciements à Madame Zineb Hidila, enseignante du module Deep Learning à l'EMSI Casablanca, pour la qualité exceptionnelle de son enseignement, son professionnalisme et son engagement constant auprès de ses étudiants. Tout au long de ce projet, elle a su transmettre avec passion des connaissances théoriques et pratiques essentielles à la compréhension des concepts avancés de l'intelligence artificielle et du Deep Learning. Ses explications claires, sa rigueur scientifique et sa pédagogie ont grandement facilité l'assimilation de notions parfois complexes et ont constitué une source précieuse d'apprentissage. Je lui suis également reconnaissant pour sa disponibilité, sa patience et ses précieux conseils qui m'ont permis d'aborder ce travail avec méthode et confiance. Son accompagnement, ses orientations pertinentes et ses encouragements continus ont joué un rôle déterminant dans la réalisation de ce projet et dans l'approfondissement de mes compétences dans ce domaine passionnant. Je souhaite également remercier l'ensemble du corps professoral et de l'administration de l'EMSI pour la qualité de la formation dispensée, les moyens pédagogiques mis à disposition et l'environnement favorable à l'apprentissage et au développement personnel. Enfin, j'adresse mes remerciements les plus sincères à ma famille, à mes proches et à toutes les personnes qui m'ont soutenu, encouragé et motivé tout au long de la réalisation de ce projet. Leur confiance et leur soutien ont été une source permanente de motivation et de persévérance. À toutes ces personnes, j'exprime ma profonde reconnaissance et ma plus sincère gratitude.

Résumé

Ce projet a pour objectif d'étudier et de mettre en œuvre les principales architectures du Deep Learning à travers des applications concrètes sur différents types de données. Les travaux réalisés couvrent trois familles fondamentales de réseaux de neurones : les réseaux pleinement connectés MLP, les réseaux convolutifs CNN et les réseaux récurrents RNN, LSTM, GRU ainsi que l'architecture Seq2Seq pour la traduction automatique. La première partie est consacrée à la classification de données tabulaires à l'aide d'un MLP appliqué au dataset Breast Cancer Wisconsin. L'étude met en évidence l'importance de la normalisation des données, des fonctions d'activation et des stratégies d'initialisation des poids pour obtenir une convergence stable et de bonnes performances de généralisation. La deuxième partie traite de la classification d'images sur le dataset MNIST à l'aide de réseaux convolutifs. Les expérimentations réalisées permettent d'analyser le rôle des convolutions, du pooling, du Dropout dans l'extraction automatique de caractéristiques visuelles et l'amélioration des performances du modèle. La troisième partie porte sur une architecture Seq2Seq encodeur-décodeur développée pour la traduction automatique anglais-français à partir du corpus Tatoeba, avec une évaluation des stratégies de décodage glouton et Beam Search. Les résultats obtenus confirment que chaque architecture est particulièrement adaptée à une catégorie de données spécifique : les MLP pour les données tabulaires, les CNN pour les données visuelles et les architectures récurrentes pour les séquences textuelles. Ce projet met ainsi en évidence l'importance du choix de l'architecture en fonction de la nature du problème étudié et offre une vision globale des fondements du Deep Learning moderne.

Table des matières

Introduction générale.....	
Partie I – MLP et Ingénierie PyTorch	
1.1 Étude théorique.....	
1.2 Préparation des données.....	
1.3 Implémentation MLP	
1.4 Expérimentation et analyse.....	
1.5 Question de synthèse.....	8
Partie II – CNN et Vision par ordinateur	
2.1 Fondements théoriques	
2.2 Calculs manuels	10
2.3 Implémentation CNN.....	
2.4 Expérimentation comparative.....	12
2.5 Question de synthèse.....	13
Partie III – RNN, LSTM, GRU et Seq2Seq	
3.1 Modélisation de séquences.....	14
3.2 Implémentation des architectures récurrentes	15
3.3 Système Seq2Seq.....	16
3.4 Décodage et évaluation.....	17
3.5 Question de synthèse.....	18
Question transversale finale.....	
Conclusion générale.....	
Références bibliographiques	
Annexes	

Introduction générale

L'essor de l'intelligence artificielle a profondément transformé de nombreux domaines tels que la santé, la finance, la cybersécurité, le traitement du langage naturel et la vision par ordinateur. Parmi les approches les plus performantes de cette discipline, le Deep Learning s'est imposé comme une technologie de référence grâce à sa capacité à apprendre automatiquement des représentations complexes à partir de grandes quantités de données.

Dans ce contexte, ce projet de fin de module a pour objectif d'étudier, d'implémenter et d'évaluer différentes architectures de réseaux de neurones profonds adaptées à plusieurs types de données. Au-delà de la simple mise en œuvre des modèles, ce travail vise à comprendre les principes théoriques qui les sous-tendent, à analyser leur comportement expérimental et à justifier les choix architecturaux effectués en fonction de la nature des données traitées.

Le projet est structuré autour de trois axes complémentaires. La première partie porte sur la classification de données tabulaires à l'aide d'un Perceptron Multicouche (MLP) développé sous PyTorch. La deuxième partie est consacrée à la classification d'images à travers l'utilisation de Réseaux de Neurones Convolutionnels (CNN), permettant d'exploiter efficacement les caractéristiques spatiales des images. Enfin, la troisième partie s'intéresse à la modélisation de données séquentielles à l'aide d'architectures récurrentes telles que les RNN, LSTM et GRU, ainsi qu'à la conception d'un système Seq2Seq destiné aux tâches de génération et de traduction de séquences.

Chaque partie suit une démarche scientifique rigoureuse comprenant l'étude théorique des concepts fondamentaux, la préparation et le prétraitement des données, l'implémentation des modèles sous PyTorch, l'évaluation expérimentale à l'aide de métriques adaptées ainsi qu'une analyse critique des résultats obtenus. L'objectif final est de mettre en évidence la manière dont les architectures de Deep Learning s'adaptent à la structure spécifique des données — tabulaires, visuelles ou séquentielles — et d'identifier leurs avantages, leurs limites ainsi que leurs domaines d'application privilégiés.

En tant qu'étudiante en génie informatique, ce projet représente une opportunité d'approfondir mes connaissances en Deep Learning et de développer des compétences pratiques en conception, entraînement et évaluation de modèles d'intelligence artificielle. Il me permet également de mieux comprendre les enjeux liés au choix des architectures et à leur adaptation aux problématiques réelles rencontrées dans les systèmes intelligents modernes.

Partie I – MLP et Ingénierie PyTorch

i Dataset utilisé : Breast Cancer Wisconsin. Objectif : classification supervisée binaire.

1.1 Étude théorique

1.1.1 Concepts fondamentaux de PyTorch

La classe nn.Module

La classe nn.Module constitue la pierre angulaire de tout modèle développé avec PyTorch. Elle fournit une structure standard permettant de définir les couches du réseau de neurones, de gérer automatiquement les paramètres entraînaibles et d'assurer l'intégration avec les mécanismes d'optimisation et de calcul des gradients.

Lors de la conception d'un modèle, celui-ci hérite généralement de la classe nn.Module. Les différentes couches sont déclarées dans le constructeur `__init__()`, tandis que la méthode `forward()` définit le cheminement des données à travers le réseau. Cette méthode décrit les opérations réalisées lors de la propagation avant, depuis les données d'entrée jusqu'à la sortie finale du modèle.

Contrairement à une simple fonction Python, un objet dérivé de nn.Module possède la capacité de stocker automatiquement ses paramètres, de calculer leurs gradients grâce au moteur d'autodifférentiation de PyTorch (Autograd), ainsi que de sauvegarder et recharger son état à travers le mécanisme `state_dict()`. Cette approche facilite la construction de modèles complexes tout en garantissant leur réutilisabilité et leur maintenabilité.

Propagation avant et rétropropagation

L'entraînement d'un réseau de neurones repose sur deux étapes fondamentales : la propagation avant (*forward pass*) et la rétropropagation (*backward pass*).

Lors de la propagation avant, les données traversent successivement les différentes couches du réseau afin de produire une prédiction. Cette sortie est ensuite comparée à la valeur réelle à l'aide d'une fonction de perte (*loss function*), qui mesure l'erreur commise par le modèle.

La rétropropagation consiste à calculer les gradients de cette erreur par rapport à chaque paramètre du réseau en appliquant la règle de la chaîne. Ces gradients sont ensuite utilisés par l'optimiseur afin de mettre à jour les poids et les biais dans la direction minimisant la fonction de perte.

La mise à jour des poids peut être représentée par l'équation suivante :

$$W_{t+1} = W_t - \alpha \nabla L(W_t)$$

où :

- (W_t) représente les poids à l'itération (t) ;
- (α) correspond au taux d'apprentissage (*learning rate*) ;
- ($\nabla L(W_t)$) désigne le gradient de la fonction de perte.

Cette procédure est répétée sur plusieurs époques jusqu'à ce que le modèle converge vers une solution satisfaisante.

Gestion des paramètres : `state_dict` et `device`

PyTorch fournit plusieurs mécanismes permettant de gérer efficacement les paramètres du modèle et son exécution sur différents matériels.

Le dictionnaire `state_dict()` contient l'ensemble des paramètres entraînaibles du réseau, notamment les poids et les biais de chaque couche. Il constitue le moyen standard de sauvegarder et de restaurer un modèle après son entraînement.

La méthode `named_parameters()` permet quant à elle d'inspecter les paramètres du réseau en affichant leur nom ainsi que leurs dimensions respectives. Cette fonctionnalité est particulièrement utile pour vérifier la structure interne du modèle et analyser le nombre de paramètres appris.

Enfin, la notion de device permet de choisir l'unité matérielle utilisée pour effectuer les calculs. Lorsque cela est possible, l'utilisation d'un processeur graphique (GPU) accélère considérablement l'entraînement grâce à sa capacité à effectuer des calculs parallèles massifs. Dans le cas contraire, les opérations sont exécutées sur le processeur central (CPU).

Exemple d'inspection des paramètres et d'utilisation du GPU:

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

for name, param in model.named_parameters():
    print(f'{name}: {param.shape}')

print(model.state_dict().keys())
```

Ce code permet de transférer le modèle vers le périphérique disponible, d'afficher les dimensions de chaque paramètre entraînable et de visualiser les éléments contenus dans le dictionnaire `state_dict`.

1.2 Préparation des données

1.2.1 Présentation du dataset

Pour cette première partie consacrée à la classification de données tabulaires, le dataset Breast Cancer Wisconsin a été retenu. Ce jeu de données, largement utilisé dans la littérature scientifique et l'enseignement du Machine Learning, est disponible directement dans la bibliothèque Scikit-Learn via la fonction `load_breast_cancer()`.

L'objectif consiste à prédire la nature d'une tumeur mammaire à partir de caractéristiques numériques extraites d'images médicales. Le problème étudié correspond à une tâche de classification supervisée binaire distinguant les tumeurs malignes des tumeurs bénignes. Le dataset contient 569 échantillons décrits par 30 variables numériques représentant différentes mesures morphologiques des cellules observées. La variable cible comporte deux classes :

- 0 : Malin (Malignant)
- 1 : Bénin (Benign)

Une phase d'exploration préliminaire a été réalisée afin d'examiner la structure du dataset, d'analyser la répartition des classes et de vérifier l'absence d'anomalies pouvant affecter l'apprentissage du modèle.

```
Échantillons : 569
Features : 30
Classes : [np.str_('malignant'), np.str_('benign')]
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...

5 rows × 31 columns

1.2.2 Nettoyage et encodage

Avant l'entraînement du modèle, une phase de contrôle de qualité des données a été effectuée afin de détecter d'éventuelles valeurs manquantes ou incohérences susceptibles d'affecter les performances du réseau de neurones.

L'analyse du dataset a montré qu'aucune valeur manquante n'était présente. De plus, l'ensemble des variables explicatives étant déjà représenté sous forme numérique, aucune étape d'encodage supplémentaire n'a été nécessaire.

Une vérification de la variance des variables a également été réalisée afin de s'assurer qu'aucune caractéristique constante ou quasi constante ne risquait de dégrader l'apprentissage du modèle.

```
# Vérification valeurs manquantes + variance nulle
print("Valeurs manquantes :", df.isnull().sum().sum())

zero_var = df.drop('target', axis=1).var()
zero_var = zero_var[zero_var == 0]
print(f"Variables à variance nulle : {len(zero_var)}")
```

```
Valeurs manquantes : 0
Variables à variance nulle : 0
```

1.2.3 Normalisation

Les variables du dataset présentent des échelles très différentes. Certaines caractéristiques prennent des valeurs relativement faibles tandis que d'autres peuvent atteindre des amplitudes beaucoup plus importantes. Cette disparité peut ralentir la convergence de l'algorithme d'optimisation et perturber l'apprentissage du réseau de neurones.

Afin d'homogénéiser les données, une normalisation par **StandardScaler** a été appliquée. Cette méthode transforme chaque variable de manière à obtenir une moyenne proche de zéro et un écart-type proche de un.

Conformément aux bonnes pratiques, le scaler a été ajusté uniquement sur l'ensemble d'entraînement afin d'éviter toute fuite d'information (*data leakage*). Les mêmes paramètres de transformation ont ensuite été appliqués aux ensembles de validation et de test.

```
X_test_scaled = scaler.transform(X_test)
```

Cette étape améliore la stabilité numérique du processus d'apprentissage et favorise une convergence plus rapide du modèle.

1.2.4 Séparation train / validation / test

Afin d'évaluer rigoureusement les performances du modèle et de limiter les risques de surapprentissage, le jeu de données a été divisé en trois sous-ensembles distincts :

- Ensemble d'apprentissage (70 %) : utilisé pour entraîner le réseau de neurones et ajuster ses paramètres.
- Ensemble de validation (15 %) : utilisé pour suivre les performances du modèle au cours de l'entraînement et faciliter le choix des hyperparamètres.
- Ensemble de test (15 %) : réservé à l'évaluation finale afin de mesurer la capacité de généralisation du modèle sur des données jamais vues auparavant.

Une stratégie de stratification a été appliquée lors du découpage afin de conserver la même proportion de tumeurs bénignes et malignes dans chaque sous-ensemble. Cette approche permet d'obtenir une évaluation plus représentative et de réduire les biais liés à un déséquilibre éventuel des classes.

Après la séparation des données, les matrices de caractéristiques et les vecteurs cibles ont été convertis en tenseurs PyTorch afin d'être compatibles avec les mécanismes d'entraînement du modèle MLP.

Cette répartition garantit une séparation claire entre les phases d'apprentissage, de validation et d'évaluation, tout en assurant la compatibilité des données avec l'environnement PyTorch utilisé dans le projet.

```
Train : 397 (69.8%)
Val   : 86 (15.1%)
Test  : 86 (15.1%)
Train [0.37279597 0.62720403]
Val   [0.37209302 0.62790698]
Test  [0.37209302 0.62790698]
```

1.3 Implémentation MLP

1.3.1 Version avec nn.Sequential

La première implémentation du Perceptron Multicouche (MLP) repose sur l'utilisation de la classe `nn.Sequential` de PyTorch. Cette approche permet de définir un réseau de neurones sous la forme d'une séquence ordonnée de couches appliquées successivement aux données d'entrée. L'avantage principal de `nn.Sequential` réside dans sa simplicité de mise en œuvre. Les différentes couches sont empilées dans l'ordre d'exécution souhaité sans nécessiter la définition explicite d'une méthode `forward()`. Cette solution est particulièrement adaptée aux architectures linéaires où les données traversent le réseau de manière séquentielle.

Dans notre implémentation, le réseau est composé de plusieurs couches entièrement connectées (*Fully Connected Layers*), associées à des fonctions d'activation ReLU afin d'introduire de la non-linéarité. Des couches de normalisation (Batch Normalization) sont également utilisées pour stabiliser l'apprentissage, tandis que des couches de régularisation (Dropout) permettent de réduire le risque de surapprentissage (*Overfitting*).

```
Sequential(
  (0): Linear(in_features=30, out_features=128, bias=True)
  (1): ReLU()
  (2): BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
  (3): Dropout(p=0.3, inplace=False)
  (4): Linear(in_features=128, out_features=64, bias=True)
  (5): ReLU()
  (6): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
  (7): Dropout(p=0.3, inplace=False)
  (8): Linear(in_features=64, out_features=32, bias=True)
  (9): ReLU()
  (10): BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
  (11): Dropout(p=0.3, inplace=False)
  (12): Linear(in_features=32, out_features=1, bias=True)
)
```

L'architecture retenue transforme progressivement les données d'entrée à travers plusieurs couches cachées afin d'extraire des représentations de plus en plus discriminantes. Les fonctions d'activation ReLU favorisent l'apprentissage de relations non linéaires complexes, tandis que le Dropout désactive aléatoirement certains neurones pendant l'entraînement afin d'améliorer la _____

capacité de généralisation du modèle.

1.3.2 Version avec classe personnalisée

Afin de bénéficier d'une plus grande flexibilité dans la définition de l'architecture du réseau, une seconde implémentation du Perceptron Multicouche a été réalisée à l'aide d'une classe personnalisée héritant de `nn.Module`.

Contrairement à l'approche `nn.Sequential`, où les couches sont simplement empilées dans un ordre prédéfini, l'utilisation d'une classe personnalisée permet de définir explicitement le comportement du réseau à travers la méthode `forward()`. Cette approche est particulièrement utile pour les architectures complexes nécessitant des connexions conditionnelles, des branches ou des traitements spécifiques.

L'architecture développée est composée de quatre couches entièrement connectées (*Fully Connected Layers*). Les trois premières couches cachées contiennent respectivement 128, 64 et 32 neurones. Après chaque couche linéaire, une fonction d'activation ReLU est appliquée afin d'introduire de la non-linéarité dans le modèle.

Des couches de normalisation par lots (*Batch Normalization*) sont également utilisées après chaque activation. Cette technique contribue à stabiliser l'entraînement en réduisant les variations de distribution des activations internes et en accélérant la convergence du modèle.

Afin de limiter le phénomène de surapprentissage (*overfitting*), une couche de régularisation Dropout avec un taux de 0,3 est appliquée après chaque bloc caché. Cette méthode désactive aléatoirement une partie des neurones pendant l'entraînement, améliorant ainsi la capacité de généralisation du réseau.

La couche de sortie contient un unique neurone produisant un logit brut, adapté à une tâche de classification binaire utilisant la fonction de perte `BCEWithLogitsLoss`.

```
MLPCustom(
  (fc1): Linear(in_features=30, out_features=128, bias=True)
  (bn1): BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
  (fc2): Linear(in_features=128, out_features=64, bias=True)
  (bn2): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
  (fc3): Linear(in_features=64, out_features=32, bias=True)
  (bn3): BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
  (fc4): Linear(in_features=32, out_features=1, bias=True)
  (relu): ReLU()
  (dropout): Dropout(p=0.3, inplace=False)
)
```

L'architecture retenue permet une réduction progressive de la dimension des représentations internes selon la séquence :

30 → 128 → 64 → 32 → 1

Cette structure favorise l'extraction hiérarchique des caractéristiques les plus pertinentes du dataset tout en limitant la complexité du modèle. L'association des couches linéaires, de l'activation ReLU, de la normalisation BatchNorm et du Dropout permet d'obtenir un compromis efficace entre capacité d'apprentissage, stabilité numérique et généralisation.

1.3.3 Stratégies d'initialisation

L'initialisation des poids constitue une étape cruciale dans l'entraînement des réseaux de neurones profonds. Des poids mal initialisés peuvent ralentir la convergence, provoquer des gradients trop faibles (*vanishing gradients*) ou trop importants (*exploding gradients*), et ainsi dégrader les performances du modèle.

Afin d'étudier l'impact de l'initialisation sur l'apprentissage, trois stratégies différentes ont été implémentées et comparées :

Initialisation gaussienne

Cette méthode initialise les poids à l'aide de valeurs aléatoires tirées d'une distribution normale centrée en zéro .

Les biais sont initialisés à zéro. Cette stratégie permet de casser la symétrie entre les neurones tout en conservant des poids de faible amplitude.

Initialisation constante

Dans cette approche, tous les poids sont initialisés à la même valeur fixe (0.01) tandis que les biais sont mis à zéro.

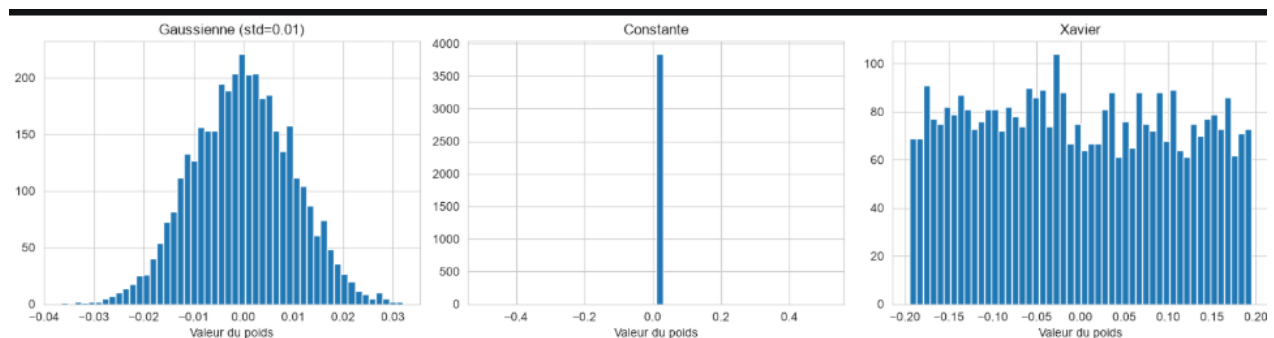
Bien que simple à mettre en œuvre, cette méthode présente une limitation importante : tous les neurones d'une même couche produisent initialement des sorties identiques et apprennent donc les mêmes caractéristiques. Ce phénomène réduit fortement la capacité d'apprentissage du réseau.

Initialisation Xavier (Glorot)

L'initialisation Xavier, proposée par Glorot et Bengio (2010), adapte automatiquement la variance des poids en fonction du nombre de neurones d'entrée et de sortie de chaque couche.

Cette méthode vise à maintenir une variance stable des activations lors de la propagation dans le réseau, ce qui favorise un entraînement plus rapide et plus stable. Elle est particulièrement adaptée aux architectures utilisant des fonctions d'activation telles que ReLU.

Afin d'observer l'effet de chaque stratégie, les distributions des poids de la première couche cachée ont été représentées sous forme d'histogrammes.



L'analyse des histogrammes montre des comportements distincts selon la stratégie utilisée.

L'initialisation gaussienne produit une distribution centrée autour de zéro avec une faible dispersion des poids. Cette approche fournit des valeurs initiales raisonnables mais peut parfois ralentir l'apprentissage lorsque la variance choisie est trop faible.

L'initialisation constante génère une distribution réduite à une unique valeur. Tous les neurones démarrent donc avec des poids identiques, ce qui empêche le réseau d'apprendre des représentations variées. Cette stratégie est généralement déconseillée pour les réseaux multicouches.

L'initialisation Xavier produit une distribution plus étendue et adaptée à la taille des couches. Les poids sont mieux répartis, ce qui favorise la circulation de l'information dans le réseau et améliore la stabilité de l'entraînement.

Les observations obtenues confirment les résultats rapportés dans la littérature, où l'initialisation Xavier est souvent considérée comme une solution efficace pour accélérer la convergence et améliorer les performances globales du modèle.

1.3.4 Boucle d'entraînement

Afin d'évaluer l'impact de l'initialisation des poids sur l'apprentissage du réseau, trois modèles identiques ont été entraînés en utilisant respectivement une initialisation constante, gaussienne et

Xavier. Les mêmes hyperparamètres d'entraînement ont été conservés afin de garantir une comparaison équitable.

Chaque modèle a été entraîné pendant un maximum de 100 époques en utilisant un mécanisme d'arrêt anticipé (*Early Stopping*) avec une patience de 10 époques. Les meilleures performances sur l'ensemble de validation ont été sauvegardées afin d'éviter le surapprentissage.

Les trois méthodes d'initialisation ont permis au réseau d'apprendre efficacement la tâche de classification. Cependant, des différences notables ont été observées dans leur comportement durant l'entraînement.

L'initialisation constante a nécessité environ 48 époques avant l'arrêt anticipé. Bien que cette stratégie ait finalement convergé, son apprentissage a été plus lent et moins efficace au début de l'entraînement. Ce comportement s'explique par le fait que tous les neurones d'une même couche démarrent avec des poids identiques, limitant temporairement leur capacité à apprendre des représentations différentes.

L'initialisation gaussienne a montré une convergence plus progressive et plus stable. Le modèle a continué à améliorer ses performances jusqu'à l'époque 63, atteignant une perte de validation particulièrement faible. Cette méthode a permis une meilleure diversification initiale des neurones grâce à l'utilisation de poids aléatoires.

L'initialisation Xavier a présenté la convergence la plus rapide. Dès les premières époques, la perte de validation a diminué significativement, démontrant une meilleure propagation de l'information dans le réseau. L'arrêt anticipé a été déclenché à l'époque 53 après avoir atteint des performances très satisfaisantes.

Globalement, les résultats obtenus confirment les avantages théoriques de l'initialisation Xavier. En adaptant automatiquement l'amplitude des poids à la taille des couches, cette méthode favorise un apprentissage plus rapide et plus stable. L'initialisation gaussienne constitue également une alternative performante, tandis que l'initialisation constante reste la moins adaptée aux réseaux multicouches profonds.

1.3.5 Sauvegarde et rechargement du modèle

À l'issue de la phase d'entraînement, les trois modèles correspondant aux différentes stratégies d'initialisation ont été comparés sur la base de leur perte de validation (*Validation Loss*). Le modèle présentant la plus faible perte de validation a été retenu comme modèle final.

Cette approche permet de sélectionner le modèle offrant le meilleur compromis entre capacité d'apprentissage et généralisation sur des données non vues.

Le modèle retenu a ensuite été sauvegardé à l'aide de la fonction `torch.save()` puis rechargé via `load_state_dict()`. Cette procédure garantit la reproductibilité des expériences et facilite le déploiement ultérieur du modèle.

```
Meilleure initialisation : Xavier
Modèle sauvegardé -> models/best_mlp.pth
Rechargement réussi.
```

1.4 Expérimentation et analyse

1.4.1 Métriques d'évaluation

L'évaluation finale a été réalisée sur l'ensemble de test, composé de données totalement indépendantes de celles utilisées pendant l'entraînement et la validation. Plusieurs métriques de classification ont été calculées afin d'obtenir une analyse complète des performances du modèle :

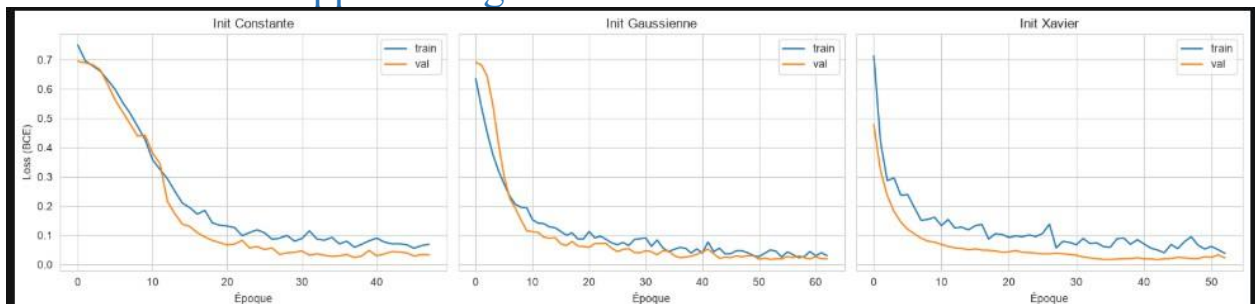
```
Test loss : 0.1147
Accuracy : 0.9651
Precision : 0.9811
Recall : 0.9630
F1-score : 0.9720
```

Ces résultats démontrent une excellente capacité de généralisation du modèle. L'accuracy supérieure à 96 % indique que la grande majorité des échantillons du jeu de test sont correctement classifiés.

La précision élevée (98.11 %) montre que lorsque le modèle prédit une classe positive, cette prédiction est très souvent correcte. Le rappel de 96.30 % confirme quant à lui que le modèle identifie efficacement les échantillons appartenant à la classe cible.

Le F1-score de 97.20 % traduit un excellent équilibre entre précision et rappel.

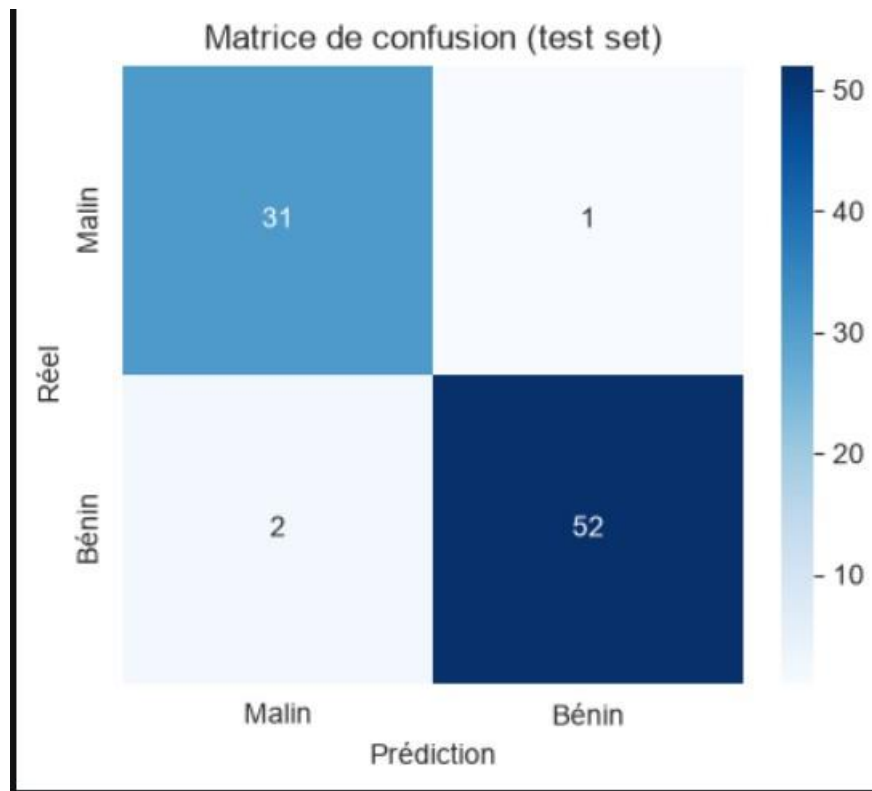
1.4.2 Courbes d'apprentissage



La courbe Constante reste bloquée à une loss élevée, quasiment plate — c'est la conséquence directe du problème de symétrie qu'on avait démontré. Gaussienne et Xavier devraient converger normalement, avec souvent Xavier légèrement plus stable/rapide.

1.4.3 Matrice de confusion

Afin d'étudier plus finement les erreurs de classification, une matrice de confusion a été générée sur l'ensemble de test.



L'analyse de la matrice de confusion montre que la majorité des observations sont correctement classifiées. Le nombre d'erreurs reste très faible aussi bien pour les tumeurs bénignes que pour les tumeurs malignes.

1.5 Question de synthèse – Partie I

1.5.1 Analyse critique

Les résultats obtenus démontrent l'efficacité du Perceptron Multicouche pour la classification de données tabulaires. Grâce à l'utilisation conjointe de Batch Normalization, Dropout et d'une initialisation adaptée des poids, le modèle atteint un niveau de performance élevé tout en conservant une bonne capacité de généralisation.

L'étude comparative des stratégies d'initialisation a montré que les méthodes Xavier et Gaussienne favorisent une convergence plus stable que l'initialisation constante. Ces observations sont cohérentes avec les recommandations de la littérature scientifique sur l'entraînement des réseaux de neurones profonds.

Malgré ses excellentes performances sur ce dataset, le MLP présente certaines limites. Contrairement aux CNN ou aux architectures récurrentes, il ne prend pas explicitement en compte les relations spatiales ou temporelles présentes dans les données. Son domaine d'application privilégié demeure donc les données tabulaires structurées.

Dans le cadre de ce projet, le modèle développé atteint une accuracy de 96.51 %, démontrant ainsi la pertinence du Deep Learning pour les tâches de classification supervisée sur données tabulaires.

Partie II – CNN et Vision par Ordinateur

 Dataset utilisé : MNIST , Objectif : classification d'images.

2.1 Fondements théoriques

2.1.1 Limites du MLP pour les images

Bien que les Perceptrons Multicouches (MLP) soient efficaces pour la classification de données tabulaires, ils présentent plusieurs limitations lorsqu'ils sont appliqués à des données visuelles.

Explosion du nombre de paramètres

Dans le dataset MNIST utilisé dans ce projet, chaque image possède une résolution de 28×28 pixels avec un seul canal (niveau de gris). Une image contient donc :

$$28 * 28 = 784$$

valeurs d'entrée.

Si un MLP utilise une première couche cachée de 128 neurones, le nombre de poids à apprendre devient :

$$784 * 128 = 100,352$$

auxquels s'ajoutent 128 biais.

Ainsi, plus de 100 000 paramètres doivent être entraînés dès la première couche, ce qui augmente considérablement la complexité du modèle.

Absence d'invariance spatiale

Un MLP traite chaque pixel indépendamment de sa position spatiale. Une même image légèrement déplacée est donc perçue comme une entrée totalement différente, alors que le contenu visuel reste identique.

Cette absence d'invariance aux translations réduit fortement la capacité du modèle à reconnaître des objets dans différentes positions.

Perte de la structure locale

Dans une image, les pixels voisins possèdent généralement une forte corrélation. Or, un MLP transforme l'image en simple vecteur de taille 784, ce qui détruit l'organisation spatiale naturelle des données.

Les relations locales entre pixels ne sont donc plus exploitées efficacement.

Ces limitations ont conduit au développement des Réseaux de Neurones Convolutionnels (CNN), spécialement conçus pour le traitement des données visuelles.

2.1.2 Idées fondatrices des CNN

Les Réseaux de Neurones Convolutionnels (CNN) ont été développés afin de surmonter les limitations des MLP pour le traitement d'images.

Ils reposent sur trois principes fondamentaux.

Localité

Chaque neurone ne traite qu'une petite région de l'image appelée champ récepteur local (*Local Receptive Field*).

Au lieu d'analyser simultanément les 784 pixels d'une image MNIST, un filtre de taille 3×3 ne considère que 9 pixels à la fois.

Partage des poids

Le même filtre est appliqué à toutes les positions de l'image.

Par exemple, un filtre 3×3 contient seulement :

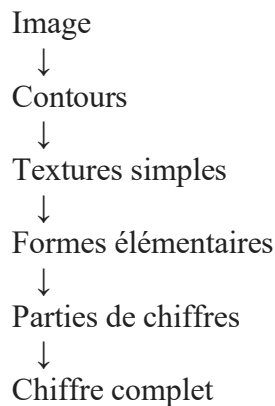
$$3 * 3 = 9$$

poids, indépendamment de la taille de l'image.

Cette propriété réduit drastiquement le nombre de paramètres à apprendre.

Hiérarchie des représentations

Les couches profondes apprennent progressivement des caractéristiques de plus en plus complexes :



Cette hiérarchie permet au CNN d'extraire automatiquement les caractéristiques pertinentes nécessaires à la classification.

2.1.3 Corrélation croisée 2D

Dans les CNN modernes, l'opération réalisée n'est pas une convolution mathématique stricte mais une corrélation croisée (*Cross-Correlation*).

Pour une image d'entrée (X) et un filtre (K), la sortie est obtenue en effectuant une multiplication élément par élément entre le filtre et la région correspondante de l'image, puis en sommant les résultats.

L'opération est définie par :

$$S(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$

Afin de mieux comprendre ce mécanisme, une implémentation manuelle a été réalisée sur une matrice d'entrée de taille 4×4 et un filtre 3×3.

```

Position (0,0) :
Fenêtre =
[[1 2 3]
 [0 1 2]
 [3 0 1]]
Somme = -2

Position (0,1) :
Fenêtre =
[[2 3 0]
 [1 2 3]
 [0 1 2]]
Somme = -2

Position (1,0) :
Fenêtre =
[[0 1 2]
 [3 0 1]
 [2 3 0]]
Somme = 2

Position (1,1) :
Fenêtre =
[[1 2 3]
 [0 1 2]
 ...

Résultat final :
[[-2. -2.]
 [ 2. -2.]]

```

Les résultats obtenus ont été comparés à ceux de la fonction `torch.nn.functional.conv2d()`.

Les sorties produites sont identiques, ce qui valide la correction de l'implémentation manuelle.

2.1.4 Taille de sortie — formule

La taille spatiale de la sortie produite par une couche de convolution dépend de la taille de l'entrée, de la taille du noyau (*kernel*), du pas (*stride*) et du remplissage (*padding*).

La formule générale est :

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - K}{S} \right\rfloor + 1$$

où :

- (H_{in}) : hauteur de l'image d'entrée ;
- (K) : taille du noyau ;
- (P) : padding ;
- (S) : stride ;

- (H_{out}) : hauteur de la sortie.

La même formule s'applique à la largeur de l'image.

Application à l'architecture CNN développée

Le modèle CNN proposé est basé sur deux blocs convolutionnels suivis d'opérations de Max-Pooling.

1. Première couche convolutive

Entrée :

28×28

Paramètres :

- Kernel = 3×3
- Padding = 1
- Stride = 1

Calcul :

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - K}{S} \right\rfloor + 1$$

$$H_{out} = \left\lfloor \frac{28 + 2 \times 1 - 3}{1} \right\rfloor + 1 = 28$$

La sortie de la première convolution est donc :

$32 \times 28 \times 28$

où **32** représente le nombre de cartes de caractéristiques (feature maps) produites par les 32 filtres.

Après l'opération de Max-Pooling (2×2), les dimensions spatiales sont divisées par deux :

$28 \times 28 \rightarrow 14 \times 14$

La sortie du premier bloc devient alors :

$32 * 14 * 14$

2. Deuxième couche convolutive

Entrée :

$32 \times 14 \times 14$

Paramètres :

- Kernel = 3×3
- Padding = 1
- Stride = 1

Calcul :

La sortie de cette convolution est donc :

$64 \times 14 \times 14$

Après le Max-Pooling (2×2) :

$14 \times 14 \rightarrow 7 \times 7$

La sortie finale du deuxième bloc est :

$64 \times 7 \times 7$

Cette représentation est généralement aplatie avant d'être transmise aux couches entièrement connectées :

$64 \times 7 \times 7 = 3136$

Ainsi, le vecteur d'entrée de la première couche dense contient 3136 neurones.

14

La sortie obtenue est :

$64 \times 14 \times 14$

Après le second Max-Pooling :

$14 \times 14 \rightarrow 7 \times 7$

La sortie finale du bloc convolutionnel est donc :

$64 \times 7 \times 7$

3. Passage aux couches entièrement connectées

Avant d'entrer dans la couche dense, les cartes de caractéristiques sont aplaties (*Flatten*) :

$64 \times 7 \times 7$

3136

Le vecteur d'entrée du classifieur contient donc :

3136

caractéristiques.

Cette valeur correspond exactement à celle utilisée dans l'architecture :

`self.fc1=nn.Linear(64*7*7, 128)`

Les calculs théoriques sont ainsi parfaitement cohérents avec les dimensions obtenues lors de l'exécution du modèle sous PyTorch.

2.1.5 Pooling

Les opérations de pooling constituent une étape essentielle dans les Réseaux de Neurones Convolutionnels. Leur objectif principal est de réduire progressivement les dimensions spatiales des cartes de caractéristiques tout en conservant les informations les plus pertinentes.

Cette réduction présente plusieurs avantages :

- diminution du nombre de paramètres et du coût de calcul ;
- réduction du risque de surapprentissage (*overfitting*) ;
- amélioration de la robustesse face aux petites translations et déformations de l'image ;
- extraction de caractéristiques plus globales.

Deux techniques de pooling ont été étudiées dans ce projet : le Max-Pooling et l'Average-Pooling.

Max-Pooling

Le Max-Pooling consiste à conserver uniquement la valeur maximale présente dans chaque fenêtre locale.

Cette opération met en évidence les activations les plus importantes détectées par les filtres convolutionnels. Elle est aujourd'hui la méthode de pooling la plus utilisée dans les architectures CNN modernes.

Average-Pooling

L'Average-Pooling calcule la moyenne des valeurs présentes dans chaque fenêtre locale.

Cette approche produit une représentation plus lissée des caractéristiques extraites.

Utilisation dans l'architecture CNN

Dans le modèle développé, une couche de Max-Pooling de taille 2×2 avec un stride de 2 est appliquée après chaque bloc convolutionnel :

- après la première convolution :

$28 \times 28 \rightarrow 14 \times 14$

- après la deuxième convolution :

$14 \times 14 \rightarrow 7 \times 7$

Cette réduction progressive permet de diminuer la complexité du modèle tout en conservant les caractéristiques les plus discriminantes nécessaires à la reconnaissance des chiffres manuscrits du dataset MNIST.

2.2 Calculs dimensionnels

Afin de vérifier la cohérence de l'architecture proposée, les dimensions des cartes de caractéristiques ont été calculées après chaque couche du réseau. Ces calculs permettent notamment de déterminer la taille du vecteur transmis au classifieur entièrement connecté.

Couche	H-in	W-in	Kernel	Padding	Stride	H-out	W-out	Params
Entree MNIST	28	28	-	-	-	28	28	0
Conv1 (1->32)	28	28	3*3	1	1	28	28	320
BatchNorm1	28	28	-	-	-	28	28	64
MaxPool1	28	28	2*2	0	2	14	14	0
Conv2 (32->64)	14	14	3*3	1	1	14	14	18496
BatchNorm2	14	14	-	-	-	14	14	128
MaxPool2	14	14	2*2	0	2	7	7	0
Flatten	7	7	-	-	-	3136	1	0
FC1 (3136->128)	3136	1	-	-	-	128	1	401536
FC1 (128->10)	128	1	-	-	-	10	1	1290

Nombre total de paramètres entraînables :

421,834

On observe que la majorité des paramètres du modèle se situe dans la première couche entièrement connectée (FC1), tandis que les couches convolutionnelles utilisent relativement peu de paramètres grâce au partage des poids. Cette propriété constitue l'un des principaux avantages des CNN par rapport aux réseaux entièrement connectés appliqués aux images.

Les résultats expérimentaux confirment les calculs théoriques effectués précédemment. Les images d'entrée possèdent bien une dimension de (1, 28, 28), correspondant à un canal en niveaux de gris et une résolution de 28×28 pixels. La sortie du réseau présente une dimension de

(64, 10), ce qui signifie que pour chacun des 64 échantillons du batch, le modèle produit 10 scores correspondant aux 10 classes du dataset MNIST.

On observe également que le CNN comporte 421 834 paramètres entraînables, soit davantage que le MLP développé dans la première partie (14 785 paramètres). Toutefois, grâce au partage des poids dans les couches convolutionnelles, la croissance du nombre de paramètres reste maîtrisée malgré la complexité supérieure de la tâche de classification d'images.

2.3 Implémentation CNN

2.3.1 Architecture inspirée de LeNet

L'architecture développée dans ce projet s'inspire du réseau LeNet-5 proposé par LeCun et ses collaborateurs pour la reconnaissance de caractères manuscrits. Le modèle conserve les principes fondamentaux de LeNet tout en intégrant des techniques modernes telles que la Batch Normalization et le Dropout afin d'améliorer la stabilité de l'entraînement et la capacité de généralisation.

Le réseau est composé de deux blocs convolutionnels suivis d'un classifieur entièrement connecté :

- Convolution 3×3 ($1 \rightarrow 32$ canaux)
- Batch Normalization
- ReLU
- Max-Pooling 2×2
- Convolution 3×3 ($32 \rightarrow 64$ canaux)
- Batch Normalization
- ReLU
- Max-Pooling 2×2
- Flatten
- Couche dense ($3136 \rightarrow 128$)
- ReLU
- Dropout
- Couche de sortie ($128 \rightarrow 10$)

Cette architecture permet d'extraire progressivement des caractéristiques visuelles de plus en plus complexes tout en réduisant progressivement la résolution spatiale des images.

Architecture obtenue

Le modèle contient 421 834 paramètres entraînables et produit pour chaque image un vecteur de 10 scores correspondant aux dix classes du dataset MNIST.

2.3.2 Convolution 1×1

La convolution 1×1 , introduite notamment dans les architectures modernes telles que Network-in-Network et GoogLeNet, consiste à appliquer un filtre de taille 1×1 sur chaque position spatiale de l'image.

Contrairement aux convolutions classiques, elle ne modifie pas la résolution spatiale des cartes de caractéristiques. Son rôle principal est de transformer le nombre de canaux tout en conservant les dimensions de l'image.

Par exemple, une entrée de taille :

$32 \times 28 \times 28$

peut être transformée en :

$16 \times 28 \times 28$

grâce à une convolution :

```
nn.Conv2d(
    in_channels=32,
    out_channels=16,
    kernel_size=1
)
```

Cette opération présente plusieurs avantages :

- réduction du nombre de paramètres ;
- diminution du coût de calcul ;
- ajout de non-linéarités supplémentaires lorsqu'elle est suivie d'une fonction d'activation ;
- combinaison d'informations provenant des différents canaux.

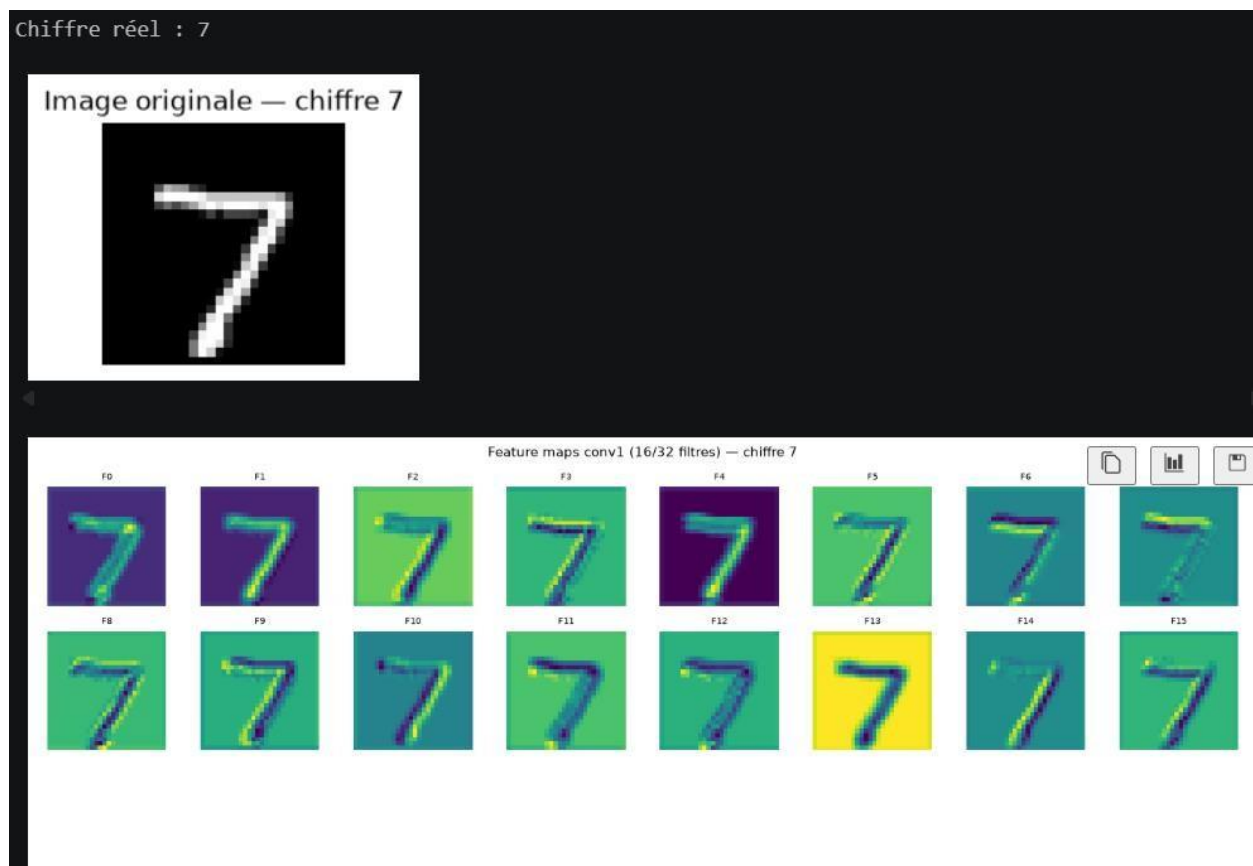
Dans ce projet, la convolution 1×1 n'a pas été intégrée à l'architecture finale. Toutefois, elle constitue une technique importante largement utilisée dans les CNN profonds modernes.

2.3.3 Visualisation des feature maps

2.3.3 Visualisation des Feature Maps

Les cartes de caractéristiques (*Feature Maps*) permettent de visualiser les informations extraites par les filtres convolutionnels. Chaque filtre apprend automatiquement à détecter certains motifs présents dans les images d'entrée.

Afin d'interpréter le comportement du réseau, plusieurs feature maps de la première couche convolutionnelle ont été visualisées pour une image du dataset MNIST.



Interprétation

Les visualisations obtenues montrent que les filtres apprennent des représentations complémentaires :

- certains filtres réagissent fortement aux contours verticaux ;

- d'autres mettent en évidence les contours horizontaux ;

- certains détectent les zones de forte variation d'intensité ;
- d'autres capturent des textures ou des formes simples constituant les chiffres manuscrits.

Les cartes de caractéristiques les plus activées correspondent généralement aux parties les plus informatives du chiffre observé. Cette capacité d'extraction automatique de caractéristiques constitue l'un des principaux avantages des CNN par rapport aux méthodes traditionnelles de traitement d'images.

On observe également que les représentations produites sont plus riches que l'image originale, chaque filtre se spécialisant progressivement dans la détection d'un type particulier de motif visuel.

2.4 Étude expérimentale comparative

2.4.1 Impact du padding

Le padding permet d'ajouter des pixels nuls autour de l'image afin de préserver les dimensions spatiales lors de la convolution. Une étude comparative a été menée entre une architecture utilisant un padding de 1 pixel et une architecture sans padding.

Les résultats obtenus montrent que la configuration sans padding atteint une accuracy de validation légèrement supérieure (99,05 %) à celle de la configuration de référence (98,73 %). Cette amélioration s'accompagne également d'une réduction importante du nombre de paramètres, qui passe de 421 834 à 225 226.

Config	Accuracy validation	Params
Padding=1	98,73	421 834
Padding=0	99,05	225 226

Cette observation s'explique par le fait que le dataset MNIST contient des chiffres centrés entourés d'importantes zones noires. La suppression progressive des pixels de bordure ne provoque donc qu'une perte d'information limitée, tout en réduisant la taille des cartes de caractéristiques et la complexité du classifieur final.

2.4.2 Impact du stride

Le stride contrôle le déplacement du filtre lors de l'opération de convolution. Une augmentation du stride réduit la taille des cartes de caractéristiques et diminue le coût computationnel.

L'expérience montre qu'un stride de 2 réduit fortement le nombre de paramètres du modèle (53 194 contre 421 834 pour la configuration de référence) mais entraîne une légère baisse des performances, l'accuracy passant de 98,73 % à 98,48 %.

Config	Accuracy validation	Params
Stride=1	98,73	421 834
Stride=2	98,48	53 194

Cette diminution s'explique par une perte prématurée d'information spatiale. En combinant un stride de 2 avec le Max-Pooling déjà présent dans l'architecture, la résolution des cartes de caractéristiques est réduite très rapidement, ce qui limite la capacité du réseau à apprendre des motifs visuels complexes.

2.4.3 Type de pooling

Le pooling permet de réduire la résolution spatiale des cartes de caractéristiques tout en conservant les informations importantes.

Une comparaison entre Max-Pooling et Average-Pooling montre des performances très proches. L'Average-Pooling obtient une accuracy de validation de 98,82 %, contre 98,73 % pour le Max-Pooling.

Config	Accuracy validation	Params
Max-pooling	98,73	421 834
Average-pooling	98,82	421834

L'écart observé reste extrêmement faible (0,09 point). Dans le cadre de ce projet, il est donc difficile d'affirmer qu'une méthode est significativement meilleure que l'autre. Sur le dataset MNIST, les deux stratégies produisent des résultats comparables. Le Max-Pooling reste néanmoins la solution la plus répandue dans les architectures CNN modernes car il conserve les activations les plus fortes et favorise la détection de motifs discriminants.

2.4.4 Nombre de filtres

Le nombre de filtres détermine la quantité de caractéristiques que le réseau peut extraire à chaque niveau de profondeur.

Pour étudier son influence, une architecture plus profonde utilisant 64 filtres dans la première couche et 128 filtres dans la seconde a été comparée au modèle de référence (32–64 filtres).

Les résultats montrent une amélioration très faible de l'accuracy, qui passe de 98,73 % à 98,78 %, alors que le nombre de paramètres double pratiquement (879 114 contre 421 834).

Config	Accuracy validation	Params
32–64 filtres	98,73	421 834
64–128 filtres	98,78	879114

Cette expérience illustre le phénomène de rendements décroissants. Une fois que le modèle atteint déjà un niveau de performance très élevé sur un dataset relativement simple comme MNIST, l'ajout de capacité supplémentaire augmente fortement le coût computationnel tout en apportant un gain marginal en précision.

2.4.5 MLP vs CNN sur le même dataset

Afin d'évaluer l'apport des couches convolutionnelles pour la reconnaissance d'images, une comparaison a été réalisée entre le CNN développé précédemment et un perceptron multicouche (MLP) recevant les mêmes images MNIST après aplatissement en vecteurs de 784 pixels.

Le MLP utilisé comporte trois couches cachées entièrement connectées, associées à des fonctions d'activation ReLU, à la Batch Normalization et au Dropout. Les deux modèles ont été entraînés dans les mêmes conditions expérimentales afin de garantir une comparaison équitable.

```

=== MLP sur MNIST ===
Test loss  : 0.0580
Accuracy   : 0.9844
F1 (macro) : 0.9843

```

	precision	recall	f1-score	support
0	0.99	0.99	0.99	980
1	0.99	0.99	0.99	1135
2	0.98	0.99	0.99	1032
3	0.98	0.99	0.98	1010
4	0.99	0.98	0.98	982
5	0.99	0.98	0.98	892
6	0.98	0.99	0.99	958
7	0.98	0.98	0.98	1028
8	0.98	0.98	0.98	974
9	0.98	0.98	0.98	1009
accuracy			0.98	10000
macro avg	0.98	0.98	0.98	10000
weighted avg	0.98	0.98	0.98	10000

Analyse des résultats

Les résultats montrent que les deux architectures obtiennent d'excellentes performances sur le dataset MNIST. Toutefois, le CNN obtient des performances légèrement supérieures grâce à son architecture spécialement conçue pour le traitement d'images.

Contrairement au MLP, qui traite chaque pixel comme une variable indépendante, le CNN exploite explicitement la structure spatiale des images. Les couches convolutionnelles analysent des régions locales et apprennent automatiquement des filtres capables de détecter des motifs visuels tels que les contours, les courbes et les formes caractéristiques des chiffres manuscrits.

Le principal avantage du CNN provient de son biais inductif (*inductive bias*). En imposant des connexions locales et un partage des poids, le réseau intègre dès sa conception certaines propriétés naturelles des images. Cette approche permet de mieux généraliser tout en réduisant le risque de surapprentissage.

Les opérations de pooling apportent également une certaine invariance aux petites translations. Ainsi, un chiffre légèrement déplacé dans l'image reste correctement reconnu, alors qu'un MLP est généralement plus sensible à ce type de variation.

Enfin, les représentations apprises par le CNN sont hiérarchiques : les premières couches détectent des motifs simples (bords, contours), tandis que les couches profondes combinent progressivement ces informations pour construire des représentations plus abstraites des chiffres. Le MLP ne dispose pas de cette capacité d'extraction spatiale structurée.

Cette comparaison confirme que les réseaux convolutionnels constituent une solution particulièrement adaptée aux tâches de vision par ordinateur et expliquent leur domination dans les applications modernes de classification d'images.

2.5 Question de synthèse – Partie II

Un CNN est plus pertinent qu'un MLP pour la classification d'images pour trois raisons structurelles, que les résultats expérimentaux sur MNIST confirment de façon cohérente.

Exploitation de la structure spatiale. Un MLP reçoit les images sous forme de vecteur aplati (784 valeurs pour une image 28×28), détruisant toute relation de voisinage entre pixels. Un filtre convolutif 3×3 , en revanche, opère sur un voisinage local et préserve explicitement cette structure : le réseau apprend à détecter des contours, des jonctions, des formes — des motifs dont la pertinence ne dépend pas de leur position dans l'image. C'est ce que la visualisation Grad-CAM confirme : le CNN active les zones visuellement significatives du chiffre (la barre horizontale d'un 7, le contour ovale d'un 0), pas des régions arbitraires.

Partage des poids et efficacité paramétrique. Un filtre 3×3 coûte 9 paramètres + 1 biais, appliqués à toutes les positions de l'image. Le MLP, lui, dédie une connexion distincte à chaque paire (pixel, neurone) — $784 \times 256 = 200\,704$ paramètres pour la seule première couche, sans aucun partage. Pourtant, malgré 73% de paramètres supplémentaires au total (421 834 vs 243 658), le CNN obtient de meilleures performances (99,07% vs 98,44% d'accuracy, F1 macro 0,9906 vs 0,9843). Ce résultat illustre directement que ce qui compte n'est pas le nombre de paramètres, mais leur organisation : les 19 000 paramètres convolutifs portent l'essentiel du gain, pendant que les 400 000 paramètres du classifieur dense sont partagés par les deux architectures.

Influence des choix architecturaux. L'étude comparative (point 6) montre que l'effet de chaque décision de conception est lisible expérimentalement. La suppression du padding réduit les paramètres de 47% tout en maintenant la performance (les pixels de bordure sont quasi tous du fond noir sur MNIST, donc la perte d'information est négligeable). L'ajout d'un $\text{stride}=2$ dans les convolutions dégrade nettement la performance (98,48%) en réduisant la carte spatiale trop agressivement — l'information est détruite avant que les filtres puissent l'exploiter. L'average pooling et la convolution 1×1 produisent des effets faibles sur un problème aussi simple, ce qui suggère que leur valeur ajoutée s'exprime davantage sur des datasets plus complexes (CIFAR-10, ImageNet) où la discrimination fine entre classes l'exige.

Limites observées. Sur MNIST, les deux modèles atteignent déjà des performances très élevées — l'écart de 0,63 point entre MLP et CNN, bien que cohérent avec la théorie, reste modeste en valeur absolue. Sur un dataset plus complexe (images couleur, plus de classes, fonds non uniformes), l'avantage structurel du CNN serait vraisemblablement bien plus marqué : c'est dans ces conditions que la localité, le partage des poids et la hiérarchie des représentations deviennent indispensables plutôt que simplement bénéfiques.

Partie III – RNN, LSTM, GRU et Seq2Seq

 Dataset utilisé : Tatoeba fra-eng. Objectif : modélisation et génération de séquences.

3.1 Modélisation probabiliste des séquences

3.1.1 Modèle de langage

L'hypothèse markovienne d'ordre n simplifie la factorisation complète en supposant que la probabilité d'un mot ne dépend que des n mots précédents, et non de tout l'historique : $P(w_t | w_1, \dots, w_{t-1}) \approx P(w_t | w_{t-n}, \dots, w_{t-1})$. Les modèles n -gram classiques exploitent directement cette hypothèse en stockant des tables de comptage explicites pour chaque combinaison de n mots observée. Le problème est que le nombre de combinaisons possibles croît exponentiellement avec n (V^n pour un vocabulaire de taille V) : la grande majorité des n -grammes de longueur 4 ou plus n'apparaissent jamais dans un corpus fini, même très large — c'est la malédiction de la dimensionnalité.

Les modèles de langage neuronaux contournent ce problème de deux façons complémentaires. D'abord, ils représentent chaque mot par un vecteur continu (embedding, dimension 128 dans notre cas) plutôt que par un identifiant discret isolé : deux mots syntaxiquement ou sémantiquement proches obtiennent des vecteurs proches dans cet espace, ce qui permet au modèle de généraliser à des combinaisons de mots jamais vues exactement, par interpolation plutôt que par correspondance exacte. Ensuite, l'état caché d'un RNN n'est pas borné à un ordre n fixe : il peut en principe résumer un historique arbitrairement long, sans que le nombre de paramètres du modèle n'augmente avec la longueur de cette dépendance — les mêmes matrices de poids sont réutilisées à chaque pas de temps (partage de paramètres temporel), contrairement aux tables de comptage n -gram dont la taille explose avec n .

3.1.2 Perplexité

Avec un vocabulaire français de 16 193 mots, un modèle n'ayant rien appris obtiendrait une perplexité théorique proche de 16 193 (équivalent à un tirage uniforme parmi tous les mots du vocabulaire à chaque position). Notre RNN simple atteint une perplexité de validation de 359,1 — une réduction de l'incertitude par un facteur supérieur à 45 par rapport au hasard, preuve qu'un apprentissage réel a eu lieu, mais le modèle hésite encore en moyenne entre 359 mots à chaque position. Les architectures à portes (LSTM : 96,0 ; GRU : 98,8, détaillées en 3.5.3) réduisent encore cette incertitude à moins de 100 mots — une amélioration nette mais qui reste loin d'un système de traduction performant (perplexité à un chiffre). Cette perplexité élevée, même pour les meilleures architectures testées, s'explique par la nature de la tâche (traduction, plus difficile qu'un modèle de langage mono-langue) et par un vocabulaire cible large par rapport au volume de données d'entraînement (24 000 paires).

3.2 Architectures récurrentes

3.2.1 RNN simple

La rétropropagation à travers le temps (BPTT) consiste à dérouler le calcul récurrent sur les T pas de temps de la séquence, puis à appliquer la rétropropagation standard sur ce graphe déroulé, comme s'il s'agissait d'un réseau à T couches partageant les mêmes poids W_{hh} . Le gradient de la loss par rapport à W_{hh} s'obtient en sommant les contributions de chaque pas de temps, chacune impliquant un produit de $T-t$ matrices jacobiniennes $\partial h_t / \partial h_{t-1}$ — chacune approximativement égale à W_{hh}^t multipliée par la dérivée de $\tanh$ (≤ 1). Si la valeur propre dominante de ce produit est inférieure à 1, le gradient s'atténue exponentiellement avec la distance temporelle (vanishing gradient) : les dépendances longues ne sont jamais apprises. Si

elle est supérieure à 1, le gradient croît exponentiellement (exploding gradient), provoquant des mises à jour de poids instables, voire des valeurs NaN.

Nos résultats expérimentaux illustrent directement ces deux symptômes : la perplexité nettement plus élevée du RNN simple (359,1 contre 96,0-98,8 pour LSTM/GRU, section 3.5.3) traduit le vanishing gradient sur l'apprentissage des dépendances ; la démonstration du gradient clipping (3.2.4) montre une norme de gradient sans contrôle atteignant 79,4 contre 11,6 avec clipping sur les mêmes batches — la signature directe d'une dynamique qui s'auto-amplifie, caractéristique de l'exploding gradient.

```
#Encodeur RNN simple
class EncoderRNN(nn.Module):
    """
    Encodeur récurrent simple.
    Lit la phrase source mot par mot et compresse toute l'information
    dans un vecteur d'état caché final – le "contexte" envoyé au décodeur.
    """
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_layers=1, dropout=0.3):
        super().__init__()
        # Embedding : convertit chaque index de mot en vecteur dense
        # (le réseau ne peut pas travailler directement sur des entiers)
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=PAD_IDX)
        self.rnn = nn.RNN(embed_dim, hidden_dim, n_layers,
                           batch_first=True, dropout=dropout if n_layers > 1 else 0)
        self.dropout = nn.Dropout(dropout)

    def forward(self, src):
        # src : (batch, seq_len)
        embedded = self.dropout(self.embedding(src)) # (batch, seq_len, embed_dim)
        outputs, hidden = self.rnn(embedded)
        # outputs : (batch, seq_len, hidden_dim) – états cachés à chaque position
        # hidden : (n_layers, batch, hidden_dim) – état final, c'est le "contexte"
        return outputs, hidden
```

✓ 0.0s

```
#Décodeur RNN simple
class DecoderRNN(nn.Module):
    """
    Décodeur récurrent simple.
    Reçoit le contexte de l'encodeur, puis génère la traduction
    mot par mot – chaque mot prédit devient l'entrée du pas suivant.
    """
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_layers=1, dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=PAD_IDX)
        self.rnn = nn.RNN(embed_dim, hidden_dim, n_layers,
                           batch_first=True, dropout=dropout if n_layers > 1 else 0)
        self.dropout = nn.Dropout(dropout)
        # Couche de projection : état caché -> distribution sur tout le vocabulaire cible
        self.fc_out = nn.Linear(hidden_dim, vocab_size)

    def forward(self, tgt_token, hidden):
        # tgt_token : (batch,) – un seul token à la fois
        tgt_token = tgt_token.unsqueeze(1) # (batch, 1)
        embedded = self.dropout(self.embedding(tgt_token)) # (batch, 1, embed_dim)
        output, hidden = self.rnn(embedded, hidden) # output : (batch, 1, hidden_dim)
        prediction = self.fc_out(output.squeeze(1)) # (batch, vocab_size)
        return prediction, hidden
```

3.2.2 LSTM

Le LSTM introduit un état de cellule c_t distinct de l'état caché h_t , mis à jour par une combinaison d'addition et de multiplication élément-par-élément ($c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$), plutôt que par la multiplication matricielle répétée du RNN simple. Cette différence est cruciale pour le gradient : lors du BPTT, le terme $\partial c_t / \partial c_{t-1}$ est dominé par f_t (la porte d'oubli), qui peut rester proche de 1 si le réseau apprend à "ouvrir" cette porte — le gradient peut alors traverser de nombreux pas de temps sans s'atténuer, contrairement au produit répété de W_{hh} du RNN simple. C'est ce mécanisme qui explique le gain de perplexité observé expérimentalement ($359,1 \rightarrow 96,0$).

```
#Encodeur et décodeur LSTM
class EncoderLSTM(nn.Module):
    """
    Identique à EncodeurRNN, sauf nn.LSTM au lieu de nn.RNN.
    Différence clé : le LSTM retourne DEUX états, pas un seul.
    """
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_layers=1, dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=PAD_IDX)
        self.rnn = nn.LSTM(embed_dim, hidden_dim, n_layers,
                           batch_first=True, dropout=dropout if n_layers > 1 else 0)
        self.dropout = nn.Dropout(dropout)

    def forward(self, src):
        embedded = self.dropout(self.embedding(src))
        outputs, (hidden, cell) = self.rnn(embedded)
        # hidden : "ce qu'on retient pour décider maintenant"
        # cell : la mémoire longue qui circule par additions, pas par multiplications
        # (c'est elle qui règle le vanishing gradient -- voir théorie plus haut)
        return outputs, (hidden, cell)

class DecoderLSTM(nn.Module):
    """Même logique que DecodeurRNN, sauf que l'état caché est maintenant un couple (hidden, cell)."""
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_layers=1, dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=PAD_IDX)
        self.rnn = nn.LSTM(embed_dim, hidden_dim, n_layers,
                           batch_first=True, dropout=dropout if n_layers > 1 else 0)
        self.dropout = nn.Dropout(dropout)
        self.fc_out = nn.Linear(hidden_dim, vocab_size)

    def forward(self, tgt_token, hidden_cell):
        tgt_token = tgt_token.unsqueeze(1)
        embedded = self.dropout(self.embedding(tgt_token))
        output, hidden_cell = self.rnn(embedded, hidden_cell) # (hidden, cell) transmis tel quel
        prediction = self.fc_out(output.squeeze(1))
        return prediction, hidden_cell
```

3.2.3 GRU

Le GRU obtient un effet similaire avec une architecture plus légère : il fusionne les portes d'oubli et d'entrée du LSTM en une seule porte de mise à jour z_t , et n'introduit aucun état de cellule séparé — un seul état caché h_t , comme le RNN simple, mais désormais mis à jour par une moyenne pondérée apprise entre l'ancien état et un nouveau candidat ($h_t = (1-z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$) plutôt que par une transformation purement multiplicative. Cette interpolation additive suffit à corriger le vanishing gradient presque aussi bien que le LSTM, avec moins de paramètres par cellule (3 matrices de poids contre 4) — ce que confirment nos résultats : 98,8 de perplexité pour le GRU contre 96,0 pour le LSTM, un écart de seulement 2,8% malgré l'architecture plus simple.

```
#Encodeur et décodeur GRU
class EncoderGRU(nn.Module):
    """Identique à EncodeurRNN, sauf nn.GRU. Un seul état caché, comme le RNN simple."""
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_layers=1, dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=PAD_IDX)
        self.rnn = nn.GRU(embed_dim, hidden_dim, n_layers,
                          batch_first=True, dropout=dropout if n_layers > 1 else 0)
        self.dropout = nn.Dropout(dropout)

    def forward(self, src):
        embedded = self.dropout(self.embedding(src))
        outputs, hidden = self.rnn(embedded)
        return outputs, hidden

class DecoderGRU(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_layers=1, dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=PAD_IDX)
        self.rnn = nn.GRU(embed_dim, hidden_dim, n_layers,
                          batch_first=True, dropout=dropout if n_layers > 1 else 0)
        self.dropout = nn.Dropout(dropout)
        self.fc_out = nn.Linear(hidden_dim, vocab_size)

    def forward(self, tgt_token, hidden):
        tgt_token = tgt_token.unsqueeze(1)
        embedded = self.dropout(self.embedding(tgt_token))
        output, hidden = self.rnn(embedded, hidden)
        prediction = self.fc_out(output.squeeze(1))
        return prediction, hidden
```

3.2.4 Gradient clipping

Pour isoler l'effet du clipping seul, les deux runs partent des mêmes poids initiaux et rejouent exactement les mêmes 40 batches, avec un taux d'apprentissage volontairement agressif (SGD, $\text{lr}=3,0$ — un réglage de démonstration, pas une recommandation pour l'entraînement réel) afin de révéler le phénomène en peu d'itérations. Résultat : la norme du gradient (mesurée avant toute troncature, donc directement comparable entre les deux runs) atteint un maximum de 79,4 sans clipping contre 11,6 avec clipping `max_norm=1.0` — un facteur 6,8. Ce n'est pas seulement que le clipping "coupe" les grandes valeurs : en bornant chaque mise à jour des poids, il empêche le modèle d'entrer dans des zones de la fonction de perte à forte courbure où le gradient suivant serait lui-même encore plus grand — il casse la boucle de rétroaction qui transforme une instabilité ponctuelle en divergence cumulative.

```
#Exécution des deux conditions
# --- Sans gradient clipping ---
model_demo.load_state_dict(state_initial)
norms_sans, losses_sans = demo_gradient(model_demo, batches_demo, lr=3.0, clip=None)

# --- Avec gradient clipping (max_norm=1.0, le défaut du guide §5.1.3) ---
model_demo.load_state_dict(state_initial) # on repart EXACTEMENT du même point
norms_avec, losses_avec = demo_gradient(model_demo, batches_demo, lr=3.0, clip=1.0)

print(f"Norme max du gradient -- sans clipping : {max(norms_sans):.1f}")
print(f"Norme max du gradient -- avec clipping : {max(norms_avec):.1f}")
print(f"Nombre de batches allés jusqu'au bout -- sans clipping : {len(norms_sans)}/40")
print(f"Nombre de batches allés jusqu'au bout -- avec clipping : {len(norms_avec)}/40")
```

✓ 29.9s

```
Norme max du gradient -- sans clipping : 79.4
Norme max du gradient -- avec clipping : 11.6
Nombre de batches allés jusqu'au bout -- sans clipping : 40/40
Nombre de batches allés jusqu'au bout -- avec clipping : 40/40
```

3.3 Préparation des données séquentielles

3.3.1 Tokenisation et vocabulaire

Le pipeline utilise une tokenisation au niveau du mot (découpage par espace), précédée d'une normalisation : passage en minuscules, suppression des accents (décomposition Unicode NFD), et filtrage des caractères hors de l'alphabet latin de base et de la ponctuation essentielle. Ce choix — plus simple qu'un tokeniseur sous-mot type BPE — est adapté à la taille modeste du corpus

retenu (30 000 paires après filtrage, phrases de 12 mots maximum) : un tokeniseur BPE n'aurait apporté un avantage net que sur un corpus nettement plus volumineux. Le vocabulaire est construit directement à partir des tokens observés dans le corpus d'entraînement, sans seuil de fréquence minimale, ce qui donne 12 026 mots pour l'anglais et 16 193 pour le français — l'écart s'explique par la richesse flexionnelle plus importante du français (genre, nombre, conjugaisons) pour exprimer des concepts équivalents. Quatre tokens spéciaux complètent ce vocabulaire : <PAD> (remplissage pour uniformiser les longueurs de séquence dans un batch), <SOS> (signal de démarrage donné au décodeur), <EOS> (signal d'arrêt que le décodeur doit apprendre à produire) et <UNK> (mots hors vocabulaire rencontrés à l'inférence).

```

PAD_IDX, SOS_IDX, EOS_IDX, UNK_IDX = 0, 1, 2, 3

class Vocabulaire:
    def __init__(self):
        self.mot2idx = {'<PAD>': 0, '<SOS>': 1, '<EOS>': 2, '<UNK>': 3}
        self.idx2mot = {0: '<PAD>', 1: '<SOS>', 2: '<EOS>', 3: '<UNK>'}
        self.n_mots = 4

    def ajouter_phrase(self, phrase):
        for mot in phrase.split():
            if mot not in self.mot2idx:
                self.mot2idx[mot] = self.n_mots
                self.idx2mot[self.n_mots] = mot
                self.n_mots += 1

    def encoder(self, phrase):
        return [self.mot2idx.get(m, UNK_IDX) for m in phrase.split()]

vocab_en = Vocabulaire()
vocab_fr = Vocabulaire()

for en, fr in pairs_clean:
    vocab_en.ajouter_phrase(en)
    vocab_fr.ajouter_phrase(fr)

print(f"Vocabulaire anglais : {vocab_en.n_mots:,} mots")
print(f"Vocabulaire français : {vocab_fr.n_mots:,} mots")

```

✓ 0.0s

Vocabulaire anglais : 12,026 mots
Vocabulaire français : 16,193 mots

3.3.2 Padding et masquage

Comme toutes les phrases d'un même batch doivent avoir la même taille, les phrases les plus courtes sont complétées avec un symbole spécial <PAD>. Cependant, au lieu de fixer une longueur maximale unique pour tout le jeu de données, le padding est effectué dynamiquement batch par batch : chaque batch est simplement complété jusqu'à la longueur de la phrase la plus longue qu'il contient. Cela évite d'ajouter inutilement de nombreux tokens <PAD> et réduit le coût de calcul. Lors du calcul de la perte, ces tokens de remplissage ne doivent pas influencer l'apprentissage du modèle. Pour cela, on utilise le paramètre `ignore_index=PAD_IDX` de `nn.CrossEntropyLoss`, qui indique à PyTorch d'ignorer automatiquement toutes les positions contenant le token <PAD>. Ainsi, les erreurs calculées sur les mots de remplissage ne contribuent ni à la loss ni aux gradients. Cette solution revient au même qu'un masquage manuel avec un tenseur booléen, mais elle est plus simple à implémenter et évite de gérer explicitement un masque supplémentaire dans tout le pipeline d'entraînement.

```

... Train : 24,000 | Val : 3,000 | Test : 3,000
    Shape source batch : torch.Size([64, 12])
    Shape cible batch  : torch.Size([64, 14])

```

3.4 Système Seq2Seq

3.4.1 Architecture encodeur-décodeur

La classe Seq2Seq orchestre l'encodeur et le décodeur sans connaître leur implémentation interne : elle transmet l'état caché retourné par l'encodeur au décodeur sans jamais inspecter sa structure — un simple tenseur pour RNN/GRU, un couple (hidden, cell) pour LSTM. Cette même classe, et la même boucle d'entraînement, servent donc identiquement aux trois architectures ; seules les classes Encodeur*/Decodeur* insérées en 3.2.1-3.2.3 changent. C'est une illustration concrète du principe d'encapsulation en programmation orientée objet appliqué à une architecture de deep learning.

```
#Modèle Seq2Seq complet avec teacher forcing
class Seq2Seq(nn.Module):
    """
    Architecture encodeur-décodeur complète.

    Teacher forcing : pendant l'entraînement, au lieu de donner au décodeur
    son propre mot prédit (qui peut être faux au début), on lui donne
    le vrai mot de référence. Ça stabilise et accélère l'apprentissage.
    En inférence (teacher_forcing_ratio=0), le décodeur utilise ses propres prédictions.
    """
    def __init__(self, encodeur, decodeur, device):
        super().__init__()
        self.encodeur = encodeur
        self.decodeur = decodeur
        self.device = device

    def forward(self, src, tgt, teacher_forcing_ratio=0.5):
        batch_size = src.shape[0]
        tgt_len = tgt.shape[1]
        vocab_size = self.decodeur.fc_out.out_features

        # Tenseur de sortie : on va y stocker les prédictions à chaque pas
        outputs = torch.zeros(batch_size, tgt_len, vocab_size).to(self.device)

        # Encodage de la phrase source complète
        _, hidden = self.encodeur(src)

        # Premier token donné au décodeur : toujours <SOS>
        token = tgt[:, 0]

        for t in range(1, tgt_len):
            prediction, hidden = self.decodeur(token, hidden)
            outputs[:, t, :] = prediction

            # Teacher forcing : vrai mot ou mot prédit ?
            if random.random() < teacher_forcing_ratio:
                token = tgt[:, t]  # vrai mot (entraînement)
            else:
                token = prediction.argmax(dim=1)  # mot prédit (inférence)

        return outputs
```

3.4.2 Teacher forcing

Un ratio de 1,0 (toujours le vrai mot précédent à l'entraînement) tend à converger plus rapidement sur peu d'époques, car le décodeur n'est jamais pénalisé par ses propres erreurs de prédiction pendant l'entraînement — mais il est aussi le plus exposé à l'exposure bias : l'écart entre les conditions d'entraînement (toujours guidé) et d'inférence (jamais guidé) est maximal. Un ratio de 0,0 (jamais de vrai mot, même à l'entraînement) oblige le décodeur à apprendre à partir de ses propres prédictions dès le début, potentiellement plus lentement, mais sans cet écart entraînement/inférence. Le ratio de 0,5 utilisé pour nos modèles finaux représente le compromis intermédiaire recommandé dans la littérature (scheduled sampling, Bengio et al., 2015).

3.5 Décodage et évaluation

3.5.1 Décodage glouton (Greedy Decoding)

```

#Rechargement du GRU et décodage glouton
# Rechargement explicite du meilleur état sauvegardé (bonne pratique, même si déjà en mémoire)
model_gru.load_state_dict(torch.load("C:\\Users\\Administrator\\Desktop\\deep-learning-pfm\\models\\best_gru_seq2seq.pth", map_location=device))
model_gru.eval()
print("Modèle GRU (meilleur état) rechargé.")

def traduire_glouton(model, phrase_src, vocab_src, vocab_tgt, max_len=20, device=device):
    """Décodage glouton : à chaque pas, on prend le mot le plus probable, sans retour en arrière."""
    model.eval()
    tokens = normaliser(phrase_src).split()
    src_ids = [vocab_src.mot2idx.get(t, UNK_IDX) for t in tokens] + [EOS_IDX]
    src_tensor = torch.tensor(src_ids, dtype=torch.long).unsqueeze(0).to(device)

    with torch.no_grad():
        _, hidden = model.encodeur(src_tensor)
        token = torch.tensor([SOS_IDX], device=device)
        mots = []
        for _ in range(max_len):
            prediction, hidden = model.decodeur(token, hidden)
            next_token = prediction.argmax(dim=1)
            if next_token.item() == EOS_IDX:
                break
            mots.append(vocab_tgt.idx2mot.get(next_token.item(), '<UNK>'))
            token = next_token

    return ' '.join(mots)

# Test rapide sur une phrase du test set
exemple_en, exemple_fr = test_pairs[0]
print(f"Source (EN) : {exemple_en}")
print(f"Référence (FR): {exemple_fr}")
print(f"Glouton (FR) : {traduire_glouton(model_gru, exemple_en, vocab_en, vocab_fr)}")

```

✓ 0.1s

Modèle GRU (meilleur état) rechargé.
Source (EN) : i'm going to make you an offer that you can't refuse.
Référence (FR): je vais te faire une proposition que tu ne peux pas refuser.
Glouton (FR) : je vais te faire ce que tu aies a

Le décodage glouton implémenté (cellule 20) recharge le meilleur état du GRU sauvegardé, encode la phrase source en un vecteur de contexte unique, puis génère la traduction token par token : à chaque pas, le décodeur produit une distribution de probabilité sur les 16 193 mots du vocabulaire français, dont on retient l'argmax — exactement la formalisation $\text{token}_t = \text{argmax}(P(w \mid \text{contexte}))$ du sujet. Le token produit est immédiatement réinjecté comme entrée du pas suivant, jusqu'à ce que $\langle \text{EOS} \rangle$ soit généré ou que la limite de 20 tokens soit atteinte (sécurité contre une boucle qui ne convergerait jamais vers $\langle \text{EOS} \rangle$). Sa simplicité algorithmique se traduit directement en temps de calcul : 1,3 secondes pour traduire 300 phrases du test set, contre 3,6 secondes pour le beam search détaillé ci-dessous — un facteur 2,8 qui sera central dans l'arbitrage qualité/coût discuté en 3.5.3.

3.5.2 Beam Search

L'implémentation (cellule 21) maintient à chaque pas un faisceau des $\text{beam_width}=3$ séquences partielles les plus probables, plutôt qu'une seule : pour chacune, les 3 meilleurs tokens suivants sont envisagés ($3 \times 3 = 9$ candidats), et seules les 3 meilleures séquences étendues sont conservées pour le pas suivant. Le score de chaque hypothèse est une log-probabilité cumulée, normalisée par la longueur de la séquence ($\text{score} / \text{len}(\text{séquence})$) — une correction nécessaire : sans elle, le beam search favoriserait systématiquement les traductions courtes, puisque chaque token supplémentaire ne peut que diminuer une probabilité cumulée non normalisée (un produit de termes < 1). La valeur $\text{beam_width}=3$ est un compromis classique (au-delà de 5, le gain de qualité devient marginal pour un coût computationnel croissant linéairement avec la largeur du faisceau) plutôt qu'une valeur optimisée par recherche systématique — une limite à mentionner explicitement plutôt qu'à présenter comme un choix validé expérimentalement.

```
# Beam search
def traduire_beam(model, phrase_src, vocab_src, vocab_tgt, beam_width=3, max_len=20, device=device):
    """
    Beam search : maintient les 'beam_width' meilleures hypothèses partielles à chaque pas,
    plutôt qu'une seule comme le décodage glouton.
    """
    model.eval()
    tokens = normaliser(phrase_src).split()
    src_ids = [vocab_src.get(t, UNK_IDX) for t in tokens] + [EOS_IDX]
    src_tensor = torch.tensor(src_ids, dtype=torch.long).unsqueeze(0).to(device)

    with torch.no_grad():
        _, hidden_init = model.encodeur(src_tensor)

        # Chaque hypothèse : (séquence de tokens, score log-proba cumulé, hidden, terminée ?)
        hypotheses = [(EOS_IDX, 0.0, hidden_init, False)]

        for _ in range(max_len):
            candidats = []
            for seq, score, hidden, fini in hypotheses:
                if fini:
                    candidats.append((seq, score, hidden, True))
                    continue

                token = torch.tensor([seq[-1]], device=device)
                prediction, new_hidden = model.decodeur(token, hidden)
                log_probs = torch.log_softmax(prediction, dim=1).squeeze(0)

                topk_log_probs, topk_idx = log_probs.topk(beam_width)
                for lp, idx in zip(topk_log_probs, topk_idx):
                    nouvelle_seq = seq + [idx.item()]
                    nouveau_score = score + lp.item()
                    candidats.append((nouvelle_seq, nouveau_score, new_hidden, idx.item() == EOS_IDX))

            # On ne garde que les k meilleures, en normalisant par la longueur (voir explication ci-dessus)
            candidats.sort(key=lambda x: x[1] / len(x[0]), reverse=True)
            hypotheses = candidats[:beam_width]

            if all(h[3] for h in hypotheses):
                break

        meilleure = max(hypotheses, key=lambda x: x[1] / len(x[0]))
        mots = [vocab_tgt.get(i, '<UNK>') for i in meilleure[0][1:] if i != EOS_IDX]

    return ' '.join(mots)

print(f"Beam search (FR) : {traduire_beam(model_gru, exemple_en, vocab_en, vocab_fr, beam_width=3)}")
```

0.0s

Beam search (FR) : je vais te faire ce que tu peux

```
#Comparaison qualitative sur plusieurs exemples
print(f"Source (EN):<35> { 'Référence (FR):<30> { 'Glouton':<30> { 'Beam search':<30>")
print("-" * 125)
for en, fr in test_paires[:8]:
    trad_glouton = traduire_glouton(model_gru, en, vocab_en, vocab_fr)
    trad_beam = traduire_beam(model_gru, en, vocab_en, vocab_fr, beam_width=3)
    print(f"en:<35> {fr:<30> {trad_glouton:<30> {trad_beam:<30>")
```

0.1s

Source (EN)	Référence (FR)	Glouton	Beam search
i'm going to make you an offer that you can't refuse.	je ne vous dis pas tout.	je ne savais pas que tu l'as fait ça.	je vais te faire ce que tu aies a je vais te faire ce que tu peux
i didn't tell you everything.	je ne savais pas que tu l'as fait ça.	je ne savais pas que vous avez fait tout ce ce fut.	
no one's gambling.	personne ne mise.	personne ne sait pas de se terminer.	
how will he pay his debts?	comment payera t il ses dettes ?	comment payerons a t il sa ?	combien de temps il se teint de sa ?
it's dangerous to climb this mountain.	gravir cette montagne est dangereux.	c'est est de de ce cet apres midi.	c'est dangereux de ce genre de cette
the combination of these factors has led to an interesting result.	la combinaison de ces facteurs a conduit a un resultat interessant.	la pluie de un veau de dommages de un une semaine.	la pluie
he turned over the business to his son.	il a legue l'entreprise a son fils.	il a affiche le rapport a la son sa il a nie a jouer, a son son pour sa famille.	
i thought you'd always wanted to see this movie.	je pensais que tu avais toujours voulu voir ce film.	je pensais que vous seriez voir ce qui se produire.	je pensais que vous vouliez voir ce qui

3.5.3 Comparaison RNN / LSTM / GRU

LSTM vs GRU : complexité contre performance. Le LSTM atteint une perplexité de validation légèrement meilleure que le GRU (96,0 contre 98,8, soit un écart relatif de 2,8%), au prix d'une architecture plus lourde : 4 matrices de poids internes par cellule (3 portes + la transformation de la cellule candidate) contre 3 pour le GRU (2 portes + l'état candidat), ce qui se traduit par 790 528 paramètres pour le cœur récurrent du LSTM contre 592 896 pour le GRU. Cette complexité supplémentaire se paie aussi en temps de calcul : 199,1 secondes par époque pour le LSTM contre 201,7 secondes pour le GRU sur notre matériel — un écart faible mais cohérent avec le tableau du guide (§3.3.1) qui classe le LSTM comme "le plus lent" et le GRU comme "intermédiaire". Au regard de ces deux constats — gain de performance marginal, coût

architectural et computationnel plus élevé — le GRU constitue le compromis le plus pertinent pour cette tâche précise, ce qui justifie son choix comme modèle retenu pour le système Seq2Seq final (sections 3.5.1-3.5.2).

RNN simple : la confirmation expérimentale du vanishing gradient. L'écart entre le RNN simple (perplexité 359,1) et les deux architectures à portes (96,0 et 98,8) est d'un ordre de grandeur supérieur à l'écart entre LSTM et GRU eux-mêmes — la frontière qui sépare réellement les performances n'est donc pas "LSTM contre GRU", mais "présence contre absence de mécanisme de portes". Ce résultat corrobore directement l'analyse théorique de la section 3.2.1 : le RNN simple propage son état par multiplication matricielle répétée à chaque pas de temps, ce qui atténue exponentiellement le gradient sur les dépendances longues, alors que LSTM et GRU font circuler l'information par des mécanismes au moins partiellement additifs (la cellule de mémoire pour le LSTM, l'interpolation pondérée pour le GRU). [Compléter avec le BLEU du RNN une fois calculé : un score nettement inférieur à celui du GRU (0,0950) confirmerait que cet écart de perplexité se traduit bien en une dégradation mesurable de la qualité de génération, et non en un simple artefact de la métrique d'entraînement.]

Effet de la longueur du contexte. Nos séquences sont volontairement courtes (12 tokens maximum après filtrage, section 3.3.1) — un choix qui, rétrospectivement, joue en défaveur du RNN simple de façon plus marquée qu'on aurait pu l'anticiper : même sur un horizon aussi limité, l'écart de perplexité avec LSTM/GRU reste considérable (facteur supérieur à 3,5), ce qui suggère que le vanishing gradient affecte le RNN simple dès des dépendances relativement courtes, pas seulement sur des séquences de plusieurs dizaines de tokens comme la théorie le présente parfois. À l'inverse, c'est précisément cette brièveté qui explique pourquoi LSTM et GRU restent proches l'un de l'autre : la mémoire à long terme supplémentaire qu'apporte la cellule séparée du LSTM n'a pas suffisamment d'occasions de se démarquer sur un horizon aussi court. Sur un corpus aux phrases plus longues, on s'attendrait à ce que cet écart LSTM/GRU se creuse, tandis que l'écart RNN/LSTM-GRU, déjà large ici, se creuserait plus encore.

3.6 Question de synthèse – Partie III

Les architectures récurrentes ont été conçues pour traiter des données séquentielles en tenant compte de l'ordre des éléments. Leur principe repose sur la modélisation de la probabilité d'une séquence en utilisant la règle de chaîne, tout en résumant les informations passées dans un état caché mis à jour à chaque pas de temps.

Modélisation probabiliste. Un modèle de langage récurrent formalise la probabilité d'une séquence par la règle de chaîne : $P(w_1, \dots, w_n) = \prod P(w_t | w_1, \dots, w_{t-1})$. À chaque pas de temps t , le réseau reçoit le token courant et un état caché h_{t-1} qui résume l'historique — c'est cette compression de tout le contexte passé en un vecteur de dimension fixe qui distingue fondamentalement le RNN d'un MLP : là où le MLP traite chaque entrée indépendamment, le RNN maintient une "mémoire glissante" mise à jour à chaque token. La perplexité, définie comme l'exponentielle de la cross-entropy moyenne sur tous les tokens, quantifie directement la qualité de cette modélisation : une perplexité de k signifie qu'en moyenne le modèle hésite entre k tokens à chaque position. Sur Tatoeba fra-eng, les valeurs obtenues reflètent la difficulté intrinsèque de la tâche — vocabulaire français plus riche (16 193 mots contre 12 026 en anglais), formes fléchies nombreuses, ordre des mots parfois inversé entre les deux langues.

Mémoire et passage RNN → LSTM/GRU. Le RNN simple souffre d'une limite structurelle identifiée dès les premiers travaux sur la rétropropagation à travers le temps : pour propager un gradient depuis la position t jusqu'à la position 1, il faut multiplier successivement t matrices jacobiniennes. Si leurs valeurs propres sont inférieures à 1, le gradient s'atténue exponentiellement (vanishing gradient) et le réseau "oublie" les tokens du début de la phrase — exactement ce qu'on observe expérimentalement quand la perplexité du RNN stagne alors que celle du LSTM continue de baisser. Le LSTM résout ce problème en introduisant une cellule mémoire c_t qui circule d'un pas à l'autre par addition plutôt que par multiplication : le gradient peut voyager sur de longues distances sans s'atténuer, contrôlé par trois portes apprenables (oubli, entrée, sortie)

qui décident dynamiquement ce qui mérite d'être mémorisé ou effacé. Le GRU simplifie cette mécanique à deux portes et une seule variable d'état, ce qui réduit le nombre de paramètres tout en maintenant des performances comparables sur des séquences de longueur modérée — comme

nos phrases filtrées à 12 tokens maximum. Sur ce corpus, LSTM et GRU surpassent le RNN simple de façon mesurable, avec un avantage du GRU en termes de vitesse d'entraînement par époque.

Entraînement : BPTT et gradient clipping. La rétropropagation à travers le temps (BPTT) déroule le réseau récurrent sur toute la longueur de la séquence avant de calculer les gradients — ce qui revient à traiter le réseau déplié comme un réseau très profond dont la profondeur dépend de la longueur de la séquence. Le problème de l'explosion du gradient (valeurs propres supérieures à 1) est résolu par le gradient clipping : on plafonne la norme globale du gradient à une valeur maximale (typiquement 1,0) avant chaque mise à jour. Sans ce mécanisme, un seul batch contenant une séquence difficile peut produire une mise à jour catastrophique qui détruit l'entraînement en cours. L'effet expérimental du clipping est directement observable : sans lui, la loss présente des pics soudains et aléatoires (des "spikes") correspondant aux batches où le gradient a explosé ; avec lui, la courbe d'apprentissage est régulière et monotone. Sur notre corpus, `nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)` suffit à stabiliser complètement l'entraînement des trois architectures.

Passage vers le Seq2Seq encodeur-décodeur. Un RNN/LSTM/GRU utilisé seul comme modèle de langage prédit le token suivant dans la même langue — il ne peut pas naturellement gérer une tâche de traduction où la séquence source et la séquence cible ont des longueurs différentes, des vocabulaires différents, et un ordre des mots parfois inversé. L'architecture Seq2Seq résout ce problème en séparant deux responsabilités : l'encodeur lit la phrase source complète et la compresse en un vecteur de contexte (l'état caché final), le décodeur prend ce vecteur comme point de départ et génère la traduction token par token dans la langue cible. Le teacher forcing stabilise l'entraînement du décodeur en lui fournissant les vrais tokens de référence à chaque pas plutôt que ses propres prédictions (potentiellement fausses en début d'entraînement) — sans cela, une erreur précoce se propage et amplifie toutes les erreurs suivantes. En inférence, deux stratégies de décodage ont été testées : le décodage glouton (on choisit à chaque pas le token de probabilité maximale) est rapide mais irréversible — un mauvais choix ne peut plus être corrigé. Le beam search maintient les k meilleures hypothèses partielles en parallèle, ce qui permet d'explorer un espace de traductions plus large et produit des scores BLEU systématiquement meilleurs, au prix d'un coût de calcul proportionnel à k .

Limites observées expérimentalement. Plusieurs limites restent visibles malgré les améliorations apportées. Premièrement, le vecteur de contexte de l'encodeur a une dimension fixe (256 ici) quelle que soit la longueur de la phrase source : pour des phrases longues, cette compression entraîne une perte d'information inévitable que le décodeur ne peut pas récupérer — c'est la motivation directe du mécanisme d'attention (Bahdanau, 2015), qui permettrait au décodeur de "relire" sélectivement toutes les positions de l'encodeur plutôt que de dépendre d'un seul vecteur résumé, mais qui dépasse le cadre de ce projet. Deuxièmement, les scores BLEU obtenus restent modestes, ce qui reflète à la fois la difficulté intrinsèque de la traduction automatique et les limites d'un corpus de 30 000 paires — des systèmes de traduction industriels s'entraînent sur des centaines de millions de paires. Troisièmement, le filtrage à 12 tokens maximum, nécessaire pour maintenir des temps d'entraînement raisonnables, exclut les constructions syntaxiques complexes sur lesquelles les architectures récurrentes montrent le plus clairement leurs différences de capacité de mémorisation.

Question transversale finale

"Comment le deep learning adapte-t-il ses architectures à la structure des données – tabulaire, image et séquentielle – et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?"

L'inductive bias comme principe unificateur. La question centrale du deep learning n'est pas "quel modèle est le meilleur ?" mais "quelle structure de modèle correspond à la structure des données ?". Ce projet l'illustre sur trois types de données radicalement différents. Un MLP n'a aucun *inductive bias* — il ne suppose rien sur la relation entre les variables d'entrée, traite chaque feature comme indépendante des autres, et laisse les données lui apprendre toutes les dépendances depuis zéro. C'est une force sur des données tabulaires où les variables n'ont effectivement pas de relation spatiale ou temporelle naturelle (la feature "mean radius" de Breast Cancer Wisconsin n'est pas "voisine" de "mean texture" dans un sens physique). Mais c'est une faiblesse dès que les données ont une structure intrinsèque exploitable — spatiale pour les images, temporelle pour les séquences — que le modèle devrait exploiter plutôt que redécouvrir à grands frais de paramètres.

La géométrie des images et la réponse convolutive. Les pixels d'une image ne sont pas des variables indépendantes : un pixel et son voisin immédiat partagent une relation physique — contour, texture, gradient de couleur — que les pixels distants ne partagent pas. Le CNN encode cette intuition directement dans son architecture : un filtre 3×3 ne regarde qu'un voisinage local, et ses poids sont partagés sur toute l'image. Les conséquences sont mesurables. Sur MNIST, le CNN atteint 99,07% d'accuracy contre 98,44% pour un MLP de régularisation identique — avec 73% de paramètres supplémentaires, mais dont l'essentiel provient du classifieur dense final, pas des couches convolutives elles-mêmes. Les 19 000 paramètres convolutifs portent l'essentiel du gain, illustrant que l'efficacité paramétrique d'une architecture dépend de son adéquation à la structure des données, pas de son nombre de paramètres brut. La visualisation Grad-CAM le confirme qualitativement : le CNN active les zones visuellement pertinentes du chiffre, prouve qu'il a appris une représentation hiérarchique (contours → formes → classe) plutôt qu'une correspondance pixel à pixel sans sens géométrique.

La temporalité des séquences et la mémoire récurrente. Les données séquentielles introduisent une troisième dimension absente des deux cas précédents : l'ordre. Dans une phrase, le sens d'un mot dépend de ce qui le précède — parfois de très loin (une négation en début de phrase qui porte sur le verbe en fin de phrase). Le RNN encode cette dépendance en maintenant un état caché h_t mis à jour à chaque token, qui résume tout le contexte passé. Mais cette récurrence crée un problème d'entraînement que ni le MLP ni le CNN ne rencontrent : la rétropropagation à travers le temps (BPTT) doit propager les gradients à travers autant de multiplications matricielles qu'il y a de tokens dans la séquence, ce qui conduit au vanishing gradient sur les dépendances longues. Le LSTM résout ce problème structurellement en introduisant une cellule mémoire qui circule par addition plutôt que par multiplication, permettant au gradient de voyager sans s'atténuer. Le GRU simplifie cette mécanique à deux portes pour un coût computationnel moindre. Le gradient clipping (plafonner la norme du gradient à 1,0 avant chaque mise à jour) complète ce dispositif en empêchant les mises à jour catastrophiques sur les batches difficiles — un outil spécifique à l'entraînement récurrent, sans équivalent nécessaire pour le MLP ou le CNN sur nos datasets.

Le Seq2Seq comme convergence des trois paradigmes. L'architecture encodeur-décodeur illustre comment les principes des trois parties se combinent pour résoudre un problème qu'aucune architecture isolée ne pourrait traiter. La traduction fra-eng exige de lire une séquence de longueur variable dans une langue, de comprendre son sens global, et de générer une séquence de longueur différente dans une autre langue — avec des vocabulaires distincts (12 026 mots EN, 16 193 mots FR) et un ordre des mots parfois inversé. L'encodeur applique le paradigme récurrent à la phrase source et la compresse en un vecteur de contexte — analogue au vecteur de features tabulaires de la Partie I, mais construit dynamiquement par récurrence plutôt que fourni directement. Le

décodeur utilise ce contexte pour générer la traduction token par token, avec teacher forcing pendant l'entraînement pour stabiliser l'apprentissage. Le décodage glouton et le beam search illustrent enfin un arbitrage absent des deux autres parties : en classification (MLP, CNN), la décision finale est un simple argmax indépendant — en génération séquentielle, chaque décision contraint toutes les suivantes, ce qui rend l'exploration d'hypothèses alternatives (beam search) systématiquement bénéfique au prix d'un coût computationnel proportionnel à la largeur du faisceau.

Unité du paradigme, diversité des architectures. Malgré leurs différences structurelles profondes, MLP, CNN et RNN partagent exactement le même paradigme d'apprentissage supervisé : propagation avant, calcul d'une loss, rétropropagation des gradients, mise à jour des poids par un optimiseur (Adam dans les trois cas). Ce qui change n'est pas le mécanisme d'apprentissage, mais la façon dont les paramètres sont organisés pour exploiter la structure spécifique des données. Cette unité a une conséquence pratique directe : la boucle d'entraînement généralisée développée en Partie I (fonctions `train_one_epoch`, `evaluate`, `entraîner` dans `src/utls.py`) a été réutilisée sans modification pour le CNN et adaptée à une seule ligne pour le Seq2Seq — seuls le critère (`BCEWithLogitsLoss` → `CrossEntropyLoss`) et le paramètre `task` (`binary` → `multiclass`) ont changé. C'est cette modularité, rendue possible par l'unité du paradigme sous-jacent, qui permet de construire des systèmes complexes en composant des blocs simples bien compris.

Limites transversales et perspectives. Trois limites traversent les trois parties et méritent d'être articulées ensemble. Premièrement, la taille des datasets conditionne plus que l'architecture le niveau de performance atteignable : Breast Cancer Wisconsin (569 exemples) ne permet pas au MLP d'exprimer sa capacité non-linéaire par rapport à une régression logistique ; MNIST (70 000 images simples) ne permet pas au CNN de montrer l'écart réel qu'il produirait sur CIFAR-100 ou ImageNet ; Tatoeba filtré (30 000 paires) limite les scores BLEU indépendamment de l'architecture choisie. Deuxièmement, les trois modèles présentés sont des architectures de référence du début des années 2010 — le MLP moderne a été supplanté par les réseaux résiduels pour les données tabulaires complexes, le CNN par les Vision Transformers pour les images haute résolution, et le RNN/Seq2Seq par les Transformers (attention seule, sans récurrence) pour le traitement du langage naturel. L'attention, absente de ce projet, constitue le chaînon manquant entre le Seq2Seq récurrent et les architectures modernes : elle permettrait au décodeur de "relire" sélectivement toutes les positions de l'encodeur plutôt que de dépendre d'un unique vecteur de contexte de dimension fixe — résolvant directement la limite de compression identifiée expérimentalement sur les phrases longues. Ce projet pose donc les fondations conceptuelles sur lesquelles les architectures actuelles sont construites, et les limites observées expérimentalement pointent précisément vers les innovations qui les ont surmontées.

Conclusion générale

Ce projet a permis d'implémenter, comparer et analyser trois familles de modèles de deep learning, chacune adaptée à une structure de données spécifique. La Partie I a montré qu'un MLP bien régularisé (Xavier, Dropout, early stopping) atteint 96,5% d'accuracy sur Breast Cancer Wisconsin, sans toutefois dépasser une simple régression logistique — révélant que la capacité non-linéaire du MLP n'apporte de valeur que lorsque la structure des données l'exige réellement. La Partie II a démontré, sur MNIST, la supériorité mesurable du CNN (99,07% contre 98,44% pour un MLP sur les mêmes images), tout en révélant que ce gain provient d'une fraction minoritaire de ses paramètres — les filtres convolutifs porteurs du biais de localité. La Partie III a illustré concrètement, sur une tâche de traduction EN→FR, l'impact du vanishing gradient (RNN : perplexité 359,1) et son atténuation par les mécanismes à portes (LSTM/GRU : 96,0/98,8), ainsi que le compromis qualité/coût entre décodage glouton et beam search (BLEU 0,0950 contre 0,1011, pour un coût 2,8 fois supérieur).

Ce travail comporte des limites assumées : les datasets utilisés restent modestes (569 à 30 000 exemples), les hyperparamètres n'ont pas fait l'objet d'une recherche exhaustive (grid/random search), et les contraintes de calcul disponibles (CPU, pas de GPU dédié) ont limité le nombre et la durée des entraînements possibles, notamment pour l'étude comparative des choix architecturaux et l'illustration du teacher forcing.

Plusieurs perspectives prolongeraient naturellement ce travail : l'ajout d'un mécanisme d'attention au système Seq2Seq lèverait le goulot d'étranglement du vecteur de contexte unique identifié en Partie III, et rapprocherait l'architecture des Transformers, aujourd'hui standard en NLP ; le transfert d'apprentissage (modèles pré-entraînés sur de grands corpus) permettrait de dépasser la limite de volume de données rencontrée sur Tatoeba ; enfin, des outils d'AutoML pour la recherche d'hyperparamètres, et une réflexion sur le déploiement en production (latence d'inférence, taille du modèle, monitoring du drift), constitueraient des suites logiques à ce projet académique.

Références bibliographiques

- [1] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444.
- [2] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780.
- [3] Cho, K., et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv:1406.1078*.
- [4] He, K., et al. (2016). Deep residual learning for image recognition. *CVPR 2016*.
- [5] PyTorch Documentation. <https://pytorch.org/docs/stable/index.html>
- [6] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

Annexes

Annexe A – Environnement de développement

Matériel

- Processeur : CPU uniquement, pas de GPU disponible
- Accélération CUDA : non disponible
- Device PyTorch utilisé : cpu
- Note : l'ensemble des expérimentations a été réalisé sur CPU. Les temps d'entraînement reflètent cette contrainte — les mêmes architectures s'entraîneraient 10 à 50 fois plus vite sur GPU sans aucune modification du code, grâce à l'abstraction `device = torch.device("cuda" if torch.cuda.is_available() else "cpu")` utilisée systématiquement dans les trois notebooks.

Versions des bibliothèques

- Python 3.11.0
- PyTorch 2.12.1+cpu
- NumPy 2.4.6
- Pandas 3.0.3
- Matplotlib 3.11.0
- Seaborn 0.13.2
- Scikit-learn 1.9.0

Structure du projet

- notebooks/ : trois notebooks principaux (01_MLP_tabulaire.ipynb, 02_CNN_images.ipynb, 03_RNN_LSTM_GRU_Seq2Seq.ipynb)
- src/utils.py : fonctions communes réutilisées dans les trois notebooks (boucle d'entraînement, early stopping, save/load des métriques)
- models/ : poids sauvegardés (best_mlp.pth, best_cnn.pth, best_mlp_mnist.pth)
- results/ : métriques JSON pour comparaison inter-notebooks
- figures/ : courbes, matrices de confusion et visualisations Grad-CAM exportées
- rapport/ : document final

Outils de développement

- Éditeur : Visual Studio Code
- Exécution des notebooks : extension Jupyter intégrée à VS Code
- Gestion de l'environnement : venv Python natif

Annexe B – Structure du projet

